Pamantasan ng Lungsod ng Maynila

ENHANCEMENT OF THRESHOLD-BASED ALGORITHM FOR DETECTING ABNORMAL DISK USAGE PATTERNS USING SYSTEM LOGS

A Thesis Presented to the
Faculty of the Computer Science Department
College of Information Systems and Technology Management
Pamantasan ng Lungsod ng Maynila
(University of the City of Manila)
Intramuros, Manila

In Partial Fulfillment of the Requirements for the Degree
Bachelor of Science in Computer Science (BSCS)

By:

Bacani, Patrick Jerome D.
Co, Khurt Luis L.

Prof. Ariel Antwaun Rolando C. Sison
Adviser
November 2026

Pamantasan ng Lungsod ng Maynila

APPROVAL SHEET

The thesis hereto titled

"Enhancement of Threshold-Based Alert Algorithm for Detecting Abnormal Disk Usage Patterns Using System Logs"

prepared and submitted by **Bacani,** Patrick Jerome D. and **Co,** Khurt Luis L. in partial fulfillment of the requirements for the degree of Bachelor of Science in Computer Science has been examined and is recommended for acceptance and approval for **ORAL EXAMINATION.**

_____

Prof. Raymund M. Dioses

Coordinator

PANEL OF EXAMINERS

Approved by the Committee on Oral Examination

with a grade of _____ on _____.

_____	_____
Prof. Raymund M. Dioses	**Prof. Connie Adriano**
Panel Member	Panel Member

Accepted and approved in partial fulfillment of the requirements for the degree of Bachelor of Science in Computer Science.

_____	_____
Raymund M. Dioses	**Dr. Khatalyn E. Mata**
CS Chairperson	*Dean*
College of Information Systems and Technology Management	College of Information Systems and Technology Management

TABLE OF CONTENTS

TITLE PAGE ..... **i**
APPROVAL SHEET ..... **ii**
TABLE OF CONTENTS ..... **iii**
LIST OF FIGURES ..... **iv**
LIST OF TABLES ..... **v**
Chapter ONE **6**
1.1 Background of the Study 6
1.2 Statement of the Problem 10
1.3 Objective of the Study 11
1.4 Significance of the Study 12
1.5 Scope and Limitations 13
1.6 Definition of Terms 14
Chapter TWO **18**
2.1 Related Literature 18
2.2 Related Studies 25
2.2.1 International Studies 25
2.2.2 Local Studies 30
2.3 Comparative Analysis 35
2.4. Synthesis 47
Chapter THREE **50**
3.1 Research Design 50
3.2 Proposed Algorithm and System Architecture 52
3.2.1 Proposed Algorithm 52
3.2.2 System Architecture 59
3.3 System Requirements 61
3.3.1 Hardware Requirements 61
3.3.2 Software Requirements 63
3.4 Theories, Methods, and Tools 64
3.4.1 Theories 64
3.4.2 Methods and Tools 74
Chapter FOUR **78**

4.1 Results of Thesis in General..... 78
4.2 Ground Truth Construction..... 79
4.3 Discussion of Objectives Result..... 79
4.4 Diagnostic Investigation of the Recall Gap..... 82
4.5 A Hybrid Safety-Net Refinement..... 83
4.6 Final Comparative Result..... 85
4.7 Objective 4: Contextual Log Correlation Performance..... 87
4.8 Summary of Findings..... 88
Chapter FIVE..... 89
5.1 Summary..... 89
5.2 Discussion of Conclusions..... 90
5.2.1 Conclusion of Objective 1..... 90
5.2.2 Conclusion of Objective 2..... 91
5.2.3 Conclusion of Objective 3..... 91
5.2.4 Conclusion of Objective 4..... 91
5.2.5 Overall Conclusion..... 92
5.2.6 Tradeoffs..... 92
5.2.7 Limitations of the Evidence..... 93
5.3 Recommendations..... 93
5.3.1 Refinement of the Baseline Drift Limitation..... 93
5.3.2 Expansion of the Evaluation Dataset..... 93
5.3.3 Adaptive or Learned Ceiling Values..... 94
5.3.4 Real-Time Production Validation..... 94
5.3.5 Extension to Additional System Resources..... 94
5.3.6 Recommendation for System Administrators and Organizations..... 95
5.3.7 Correlation for Prioritisation versus Suppression..... 95
REFERENCES..... 96

LIST OF FIGURES

FIGURE No. **FIGURE TITLE** **PAGE No.**

Figure 3.2.2 Data Flow Diagram of the System Architecture---------- 60
Figure 3.4.1 Dynamic Baseline vs Static Threshold Comparison---------- 69
Figure 3.4.2 Contextual Log Correlation Process Diagram---------- 73
Figure 4.1. Ground-truth labeling execution output---------- 79
Figure 4.2 Execution output theta_z sweep + Static 80%---------- 82
Figure 4.3. Execution output of the final Enhanced Threshold-Based Alert Algorithm configuration---------- 86

LIST OF TABLES

TABLE No. **TABLE TITLE** **PAGE No.**

Table 2.3.1 Comparative Analysis of International Studies..... 36
Table 2.3.2 Comparative Analysis of Local Studies.....42
Table 3.1 Hardware Requirements.....62
Table 3.2 Software Requirements.....63
Table 4.1 Z-Score Threshold Sensitivity Sweep (W = 50, no ceiling)..... 81
Table 4.2 Window-Size Sweep (θ_z = 2.0, no ceiling).....82
Table 4.3 Joint θ_z × θ_ceiling Sensitivity Sweep (selected results; 63 combinations run). 84
Table 4.4 Final Comparative Performance of Static Threshold vs. Enhanced Algorithm.85
Table 4.5 Bootstrap 95% Confidence Intervals (5,000 resamples).....86

Chapter ONE

INTRODUCTION

1.1 Background of the Study

In modern enterprise and academic IT infrastructures, effective operational monitoring relies heavily on continuous resource tracking and log analysis. Traditional infrastructure management frameworks predominantly utilize static, single-point threshold rules such as triggering high-severity alerts when disk space utilization crosses a fixed boundary (e.g., 80%). While straightforward to configure, these static detection boundaries are fundamentally limited in dynamic computing environments. They fail to account for routine statistical variance, operational cycles, or temporary workload surges, thereby serving as a primary driver of alert noise and operational inefficiency. Albert (2024) critically demonstrated that static, boundary-based detection mechanisms cannot differentiate between expected operational spikes and genuine system degradation, underscoring the necessity of transitioning toward dynamic, statistically grounded monitoring boundaries.

Within local operational settings, local enterprise and academic networks face heightened operational risks due to high-density data traffic and limited staff bandwidth. The Philippine Threat Landscape Report 2024–2025 emphasizes that excessive alert volume in local organizations regularly masks critical operational anomalies, allowing high-severity issues to remain undetected until severe performance degradation occurs (Cyberint & Check Point Philippines, 2025). To address these operational vulnerabilities, modern system architectures require dynamic threshold recalibration mechanisms that continuously adjust decision boundaries relative to incoming event streams (Lu et al., 2026). Furthermore, integrating structured log entries provides rich contextual metadata that raw metric counters lack, allowing monitoring systems to capture sequential failure signatures alongside numerical resource trends (Liu et al., 2024).

This study is based on and also expands in a number of known algorithmic underpinnings:

Static Threshold Monitoring: The current monitoring systems offer pre-defined upper-bound thresholds; the term should be raising an alarm when disk consumption exceeds 80%. Although it is easy to apply, this method creates too many false alarms when performing a valid high-usage operation like a backup or batch processing, which causes desensitization of administrators.

Sliding Window Analysis: The modern monitoring architectures in use today make use of a rolling time-window model in determining disk utilization trends over a configurable time frame. This paper improves on this by adding a dynamic baseline which changes to historical trends and differentiates between normal spikes and truly abnormal behavior.

Z-Score Anomaly Detection: To process outlier detection the current monitoring tools apply the Z-Score statistical metric- a measure of the number of standard deviations a point is off the mean. The system can also only identify statistically significant anomalies by determining the deviation between the current usage and the historical baseline.

Log Pattern Correlation: Advanced monitoring systems play the role of correlating log entries across two or more sources; the term should be application logs, kernel logs, and filesystem logs, to determine root causes of abnormal disk usage. In the case of direct threshold violations, the systems compare concurrent log entries in order to identify the concurrent spike of usage as either benign or representative of a failure condition.

Rather than the system crying wolf when there is a transient surge in disk utilization, our algorithm will combine such tactics into a unified adaptive detection

cycle. With the ability to compare the usage patterns with an active baseline and to match the characteristics of log events in real time, the system will be able to tell when the work is normal or when it is abnormal and thus turn what used to be a traditionally reactive task into an automatic, intelligent and proactive task.

As researchers, we noted that administrators waste a lot of time in investigating false warnings and ignoring or neglecting the real warning signs. This paper examines how it is possible to make alert systems smarter by incorporating an improved threshold algorithm. The algorithm will automatically contextualize the usage data relative to historical trends to generate meaningful actionable alerts instead of the system raising an alarm upon crossing every threshold.

Existing Algorithms & Their Problems

1. Static Fixed Threshold Monitoring

Step 1. Establishes one constant value (e.g., 80% disk usage) to act as an alert.

Step 2. *Issues an alarm as soon as the usage exceeds the set limit. Such a strategy is unable to discriminate between a legitimate peak (e.g. a planned backup) and a true storage crisis and therefore achieves high false positive rates.* ← **PROBLEM 1**

2. Sliding Window Analysis

Step 1. *Measures disk consumption within specified rolling time frame (e.g. 10 minutes). (No adaptive baseline- the size of the window is constant and not indicative of long-term usage, time-of-day variations and results in incorrectly classified alerts during foreseeable peak times)* ← **PROBLEM 2**

Step 2. Computes the mean usage during the window and compares it with the threshold.

3. Z-Score Anomaly Detection

Step 1. Gathers historical disk consumption data in as input.

Step 2. *Calculates the standard deviation and the mean of the whole historical dataset.* ← **PROBLEM 3** *It is computationally costly and sensitive to concept drift (past patterns can no longer be a good predictor of present normal behaviour) to compute statistics over a historical data set of unlimited size.*

Step 3. Points in flags data that are above a Z-Score as anomalies.

4. Log Pattern Correlation (Multi-Source Correlation)

Step 1. Consumes log streams across various system origin sources.

Step 2. *Matches complex regular expression on all sources of logs to determine correlations.* ← **PROBLEM 4** *(Too high level of scope - matching all log records adds too much processing overhead and adds latency to response time, and real-time detection is impossible in large production systems with huge volumes of log data.)*

Step 3. Produces correlated alert notification between the disk event and its probable cause.

1.2 Statement of the Problem

The systems used in monitoring are highly dependent on threshold-based selectors including a fixed percentage threshold, a fixed polling rate, and pre-programmed alert response rules in order to identify abnormal disk consumption. Under the conditions that these rules do not take into consideration the dynamical workload patterns or the scheduled events, the alert systems often produce the false positives when the underlying storage system is actually operating properly. An analysis of the algorithms that are available to solve this problem demonstrates that each of the solutions focuses on a particular segment of this issue and presents its own essential restrictions.

The following problems are aimed at in this study:

1. Current systems use a single fixed threshold value that does not consider time-of-day, planned workloads, or historical usage cycles. This causes the system to trigger alerts during normal spikes while missing slow, gradual disk degradation reducing overall detection accuracy.

2. Disk usage is measured only within a fixed time window, with no awareness of long-term behavioral patterns. When multiple anomalies occur at the same time, the

system has no way to rank or prioritize them, resulting in unstructured and inconsistent alert handling.

3. Statistical baselines are calculated rigidly over an entire historical dataset with no mechanism to update over time. When normal usage patterns shift due to new workloads or system upgrades, the model fails to adapt. A problem known as concept drift.

4. The system applies full cross-source log matching even to simple, isolated disk events. This excessive processing overhead makes real-time detection impractical in environments that generate large volumes of log data.

All four issues point at the root cause of the current gap in disk monitoring methods - each algorithm is in full isolation, only taking care of its own small world and being oblivious to the problems of the others. Failure of any given detection strategy will result in a failure to use any other mechanism to revert to, which results in either over-alarming or failure to detect true anomalies. Absent a cohesive, responsive, and context-sensitive detection apparatus, disk usage tracing is noisy, costly to sustain, and of diminished use in actual practice. This research hence develops an Enhanced Threshold-Based Alert Algorithm which incorporates these existing strategies into one unified detection loop turning alert generation, which is a reactive error-prone operation, to an adaptive, consistent process.

1.3 Objective of the Study

The expected result of this study is to make automated disk usage monitoring more reliable by developing a better threshold-based alert algorithm that can identify the presence of the abnormal disk use patterns on the basis of the system logs.

Specifically, this research paper will set out to:

1. Identify the most common triggers of false positive and false negative alerts in current threshold-based disk monitoring systems, particularly those caused by

fixed threshold values that fail to account for time-of-day variations, planned workloads, and historical usage cycles, in order to establish a clear baseline of detection weaknesses that the enhanced algorithm must resolve.

2. Develop an enhanced alert algorithm that incorporates a long-term behavioral baseline alongside an anomaly severity scoring mechanism, enabling the system to distinguish between simultaneous disk usage events and handle them in a structured, prioritized, and consistent manner.
3. Implement an adaptive statistical detection mechanism using sliding window dynamic baseline calculation and Z-Score anomaly scoring, allowing the algorithm to continuously update its reference point and accurately detect abnormal behavior even when normal usage patterns shift due to new workloads or system upgrades.
4. Apply a targeted contextual log correlation approach that limits cross-source log matching only to entries directly associated with detected disk metric anomalies, reducing unnecessary computational overhead and making real-time detection feasible in high-throughput, high-volume log environments.

1.4 Significance of the Study

The outcome of the study will have the following benefits:

For the System Administrators. The suggested methodology has the potential to assist the administrators to devote less time to the investigation of false alerts occurring following the alterations in the workload patterns, and to respond to the actual storage anomalies more quickly.

For the Developers. The fact that the monitoring feedback is more accurate can be beneficial to the developers in that they should be given reliable feedback when the infrastructure is being deployed and when it is being load tested so that the infrastructure problems can be realized earlier in the development cycle.

For the Students and Future Researchers. The study can become a source of information to students and researchers, who are currently working on the domains of monitoring systems, anomaly detection systems, and smart alert systems.

For the Organizations. Businesses that can use automated infrastructure monitoring can possibly enhance their operational efficiency and can save money by reducing false alerts and the amount of unplanned downtime due to detected disk failures that are not detected in time.

In general, this research will contribute to the advancement of the sphere of automated system monitoring as it offers a flexible and versatile solution to the issue of disk usage abnormality identification and notification categorization.

1.5 Scope and Limitations

The present study is dedicated to the creation of an improved threshold-based alert algorithm that will be used to monitor disk usage with the help of system logs in Linux-based server settings. The research entails the design of a prototype which can consume log data, calculate dynamic baseline and automatically categorize disk usage events as normal or abnormal. The empirical validation will be done on a curated collection of system logs and the performance of the algorithm will be measured by the metrics of precision, recall and false positive rate.

This monitoring is specially applicable to disk usage and is not applicable to other measures of system resources like CPU, memory, or network usage. It has a rule-based, statistical detection mechanism, which does not rely on deep learning or more complex machine learning models and was made based on resource constraints and the need to be interpretable and deployable. Moreover, the experimental confirmation is derived on a small set of log datasets, and this might limit the overall transference of the result into all actual production settings.

Empirical validation for this study was conducted using a synthetic disk-usage series generated by the researcher and interleaved, as diskmon entries, with the syslog lines of the Loghub Linux system log dataset (Zhu et al., 2023). The series spans 365 days at five-minute intervals (105,122 readings), and its anomalies were injected by the researcher, so the results characterize a controlled benchmark rather than production behavior. Ground truth labels required for quantitative evaluation were constructed independently of the detection algorithm's code, using a fixed critical-percentage threshold and nearby failure-keyword matching applied directly to the raw log text, as detailed in Chapter Four. The labeling threshold and the safety-net ceiling later selected share the value 85%, parameters were selected and scored on the same data, and six keyword-only anomalies cannot be detected by any usage-based method. Ground truth labels required for quantitative evaluation were constructed independently of the detection algorithm itself, using a fixed critical-percentage threshold and nearby failure-keyword matching applied directly to the raw log text, as detailed in Chapter Four. It is also theorized that the adaptive baseline mechanism, while effective at detecting sudden usage spikes, may be less sensitive to slow, gradual degradation, since a continuously updating baseline could absorb such trends as normal before they are flagged. A limitation inherent to single-baseline statistical methods.

1.6 Definition of Terms

This section provides operational definitions of important terms used in the study for clarity.

Alert Fatigue – The state in which system administrators become desensitized to monitoring alerts due to an excessive volume of false positives, resulting in delayed or missed responses to genuine system anomalies.

Anomaly Detection – A computational process for identifying data points, patterns, or events that deviate significantly from an established norm or expected behavior within a dataset.

Baseline – A computed reference value representing the normal or expected behavior of a system metric, against which current readings are compared to determine deviation.

Batch Processing – A method of running high-volume, repetitive data jobs at a scheduled time, which temporarily elevates disk utilization and is commonly misidentified as abnormal activity by static threshold systems.

Concept Drift – A phenomenon in which the statistical properties of normal system behavior change over time due to new workloads or system upgrades, causing a previously accurate detection model to become misaligned with current usage patterns.

Contextual Log Correlation – The process of cross-referencing disk metric anomalies against concurrent log entries from directly associated system processes to determine whether a detected spike represents a genuine failure or a benign event.

Detection Accuracy – A performance measure of how correctly an algorithm classifies disk usage events as either normal or abnormal, typically evaluated through precision, recall, and false positive rate metrics.

Disk Usage Pattern – The characteristic behavior of disk utilization over time within a system, including regular cycles, peak periods, and sustained trends that define what constitutes normal storage activity.

Dynamic Baseline – A continuously updated reference value that adjusts over time based on the statistical properties of recent historical usage data, enabling the system to distinguish between normal behavioral shifts and genuine anomalies.

Dynamic Threshold – An alert trigger value that adapts over time based on computed statistical properties of historical usage data, as opposed to a fixed static value.

False Negative – A missed alert that occurs when a monitoring system fails to flag a genuine disk usage anomaly, typically because gradual degradation never crosses a fixed threshold ceiling.

False Positive – An alert raised by a monitoring system for a condition that does not represent a genuine anomaly, typically caused by legitimate workload spikes being misclassified as abnormal.

Log Correlation – The process of cross-referencing related entries from one or more system log sources to identify the relationship between a detected metric anomaly and its probable causal event.

Log Pattern Correlation – An advanced monitoring technique that matches log entries across multiple system sources such as application logs, kernel logs, and filesystem logs to determine the root cause of abnormal disk usage events.

Precision – A performance metric that measures the proportion of triggered alerts that correspond to genuine anomalies, used to evaluate how accurately the algorithm avoids false positive detections.

Recall – A performance metric that measures the proportion of actual anomalies that are successfully detected by the algorithm, used to evaluate how effectively the system avoids false negatives.

Sliding Window Analysis – A temporal monitoring technique that evaluates disk utilization trends over a rolling, configurable time frame rather than at a single point in time, providing short-term behavioral context for anomaly detection.

Static Threshold Monitoring – A monitoring approach in which a single fixed value, such as 80% disk usage, serves as the alert boundary regardless of workload context, historical patterns, or time-of-day variations.

System Log – A structured or semi-structured record generated by an operating system or application that documents events, state changes, errors, and resource usage over time.

Threshold-Based Monitoring – A monitoring approach in which alerts are triggered when a measured metric exceeds or falls below a defined boundary value, either static or dynamic.

Z-Score – A statistical measure that expresses the number of standard deviations a given data point is from the mean of a reference distribution, used in this study to quantify the degree of disk usage deviation and determine whether it constitutes a statistically significant anomaly.

Chapter TWO

REVIEW OF RELATED LITERATURE AND STUDIES

This chapter covers the review of related literature and studies that serve as the foundation of the proposed study. By reviewing relevant sources, we aim to provide a comprehensive understanding of existing works and their contributions to the field of threshold-based disk usage monitoring, anomaly detection using system logs, and alert management in infrastructure systems.

2.1 Related Literature

Static Threshold Monitoring and Its Limitations

Threshold-based monitoring remains a foundational approach in system administration and infrastructure management, wherein a predefined value such as a fixed disk usage percentage triggers alerts when exceeded. While computationally simple and easy to configure, static thresholds have long been identified as a key contributor to alert fatigue in modern operational environments. Alert fatigue occurs when system administrators are inundated with a high volume of alerts, many of which are false positives generated by legitimate workload patterns such as scheduled backups, batch jobs, or software deployments that momentarily elevate disk utilization beyond the threshold boundary. Albert (2024) established through a comprehensive critique of log anomaly detection methods that static boundary-based detection rules are fundamentally limited in their ability to distinguish between genuinely abnormal behavior and expected deviations arising from natural system cycles, highlighting the necessity of statistical, context-aware detection boundaries over fixed percentage cutoffs.

In the Philippine context, organizations continue to depend heavily on static threshold configurations due to their simplicity and low operational overhead. However, the Philippine Threat Landscape Report 2024–2025 documents how alert noise in local

organizations contributes directly to overlooked system anomalies, confirming that volume-based alerting models create operational blind spots in environments with dynamic and unpredictable workloads. This makes the enhancement of the threshold mechanism a practically relevant and locally significant research direction.

Dynamic Baseline and Adaptive Threshold Approaches

To address the inadequacy of static thresholds, research has progressively shifted toward adaptive and dynamic baseline mechanisms that calibrate alert boundaries relative to historical behavioral patterns rather than fixed values. Lu, An, Chen, and Diao (2026) demonstrated through log-based anomaly modeling that incorporating a Dynamic Thresholding Prediction Module enables systems to adjust alert boundaries continuously based on short-term statistical fluctuations. Their work establishes that dynamic boundary calibration provides direct theoretical grounding for why static thresholds—such as conventional 80% disk usage alerts—fail under shifting workload conditions, proving that adaptive recalibration is essential for maintaining accurate alert sensitivity over long operational runtimes.

The concept of dynamic thresholding has received considerable attention in recent research, particularly in cloud and server environments where workloads are inherently variable. Sun, Sankararaman, and Narayanaswamy (2024) further show that anchoring dynamic threshold recalculation directly to live observation streams eliminates configuration lag, allowing detection boundaries to evolve alongside actual system behavior rather than relying on manual adjustments or static configuration files.

Anomaly Detection in System Logs

Log-based anomaly detection has emerged as a mature alternative to metric-only threshold monitoring, leveraging structured and semi-structured records produced by operating systems to provide rich contextual information about system states. Liu, Ren, Wang, and Zhou (2024) demonstrated that temporal dependencies and log event

sequences carry critical diagnostic information for identifying system failures, proving that time-aware sequence analysis significantly outperforms stateless threshold comparisons. Their framework establishes that evaluating sequential patterns in log entries enables systems to detect subtle resource anomalies that single-point metric checks fail to capture.

Al-Mousawi (2024) further contributed to this domain by demonstrating that system log data contains reliable, interpretable signatures of failure conditions when extracted directly from parsed event streams. By extracting resource metrics and operational contexts directly from structured log records rather than relying solely on raw operating system counters, log-centric architectures preserve diagnostic associations that counter-only systems discard.

Z-Score and Statistical Anomaly Quantification

The Z-Score is a widely adopted statistical measure in anomaly detection that quantifies the deviation of a given observation from the mean of a reference distribution in units of standard deviation. Liang, Zhang, Sivasubramaniam, Jette, and Sahoo (2022) applied statistical anomaly modeling in their analysis of BlueGene/L system failure patterns, demonstrating that computing deviation scores relative to historical baselines enables the reliable identification of statistically significant anomalies while naturally filtering out benign fluctuations that fall within expected variance boundaries. Their work established the principle that anomaly significance must be evaluated relative to the statistical properties of normal behavior rather than against an absolute fixed value.

Pena, de Assis, and Proenca (2021) applied this principle more directly in a rule-based intelligent system for disk usage anomaly detection, demonstrating that integrating statistical deviation analysis with log-derived rules produces detection outcomes that are substantially more accurate and operationally actionable than single-method approaches. Their system achieved improved precision by combining a

quantitative anomaly score with contextual log evidence, a design philosophy that closely parallels the architecture of the enhanced threshold-based alert algorithm developed in the present study.

A Comparative Study of Unsupervised Anomaly Detection Algorithms (2022) evaluates how different algorithms establish the boundary for abnormal behavior in large datasets, confirming through comparative analysis that statistical deviation-based methods consistently outperform fixed threshold approaches in precision while remaining computationally feasible for real-time deployment, particularly when combined with efficient data windowing strategies.

Trend-Based Drift Detection Using Exponentially Weighted Moving Averages

While Z-Score deviation from a historical baseline is effective at identifying abrupt, high-magnitude anomalies, several studies have demonstrated that this class of point-wise statistical methods is fundamentally limited in its ability to detect slow, gradual shifts in a monitored signal. Zhou and Tang (2022) directly established this limitation in their work on satellite image time series anomaly detection, demonstrating that a Z-Score-based residual method (Z-STR) could only detect anomalies with large shifts but failed to detect anomalies with small or gradual shifts. To address this gap, they proposed a method based on the Exponentially Weighted Moving Average of the same residuals (EWMA-STR), showing that by incorporating information from all historical time series data rather than relying on a single snapshot deviation, the EWMA-based approach could detect both abrupt and gradual anomalies where the Z-Score-based method alone could not.

This finding has been reinforced in more recent monitoring literature. Hoblos (2025) introduced AnEWMA, a lightweight and computationally efficient anomaly detection approach that leverages the responsiveness of EWMA to subtle shifts in data trends, noting that traditional methods relying on fixed control limits often produce a high

rate of false positives or missed anomalies, particularly in the presence of noisy or non-stationary data, a finding directly relevant to the false-positive and false-negative failure modes documented in Section 1.2 of the present study. Similarly, Louw, Malela-Majika, Adekeye, and Saghir (2026) proposed a windowed mean drift EWMA (WMD-EWMA) control chart for monitoring autocorrelated processes, building on the established principle that combining a windowing mechanism with EWMA smoothing improves sensitivity to sustained mean drift while helping to control the false alarm rate that a single EWMA statistic alone can produce under autocorrelated conditions.

Taken together, these studies establish that EWMA-based trend detection is a well-supported complement to, rather than a replacement for, point-wise deviation methods such as Z-Score, since the two techniques are sensitive to different classes of anomalies. This directly informs the design of the present study's enhanced algorithm, which incorporates a trend-based drift detection component alongside its existing Z-Score mechanism specifically to address the gradual degradation detection gap identified in Section 1.2.

Sliding Window Analysis and Temporal Context

Sliding window analysis represents a temporal enhancement to threshold monitoring that evaluates metric behavior over a rolling time interval rather than at a single point in time. Yuan, Park, Huang, Liu, Lee, Tang, Zhou, and Savage (2023) investigated the relationship between proactive logging strategies and failure diagnosis efficiency, demonstrating that temporal context derived from log entry sequences provides substantially richer information for root cause analysis than instantaneous metric snapshots. Their work on conservative proactive logging practices informed the understanding that pre-anomaly log patterns carry diagnostic value that reactive, single-point detection misses.

The Temporal Logical Attention Network for Log-Based Anomaly Detection (2024) explores how to identify abnormalities in distributed systems where disk and resource usage behave non-linearly across time. The study confirms that time-aware detection frameworks that incorporate sequential dependencies between log events outperform stateless threshold comparisons, validating the inclusion of sliding window mechanisms within the enhanced detection architecture proposed by the present study.

Multi-Source Log Correlation

Advanced monitoring systems correlate log entries from multiple sources to identify causal relationships between observable events and metric anomalies. Zhao, Chen, Wang, Peng, Wang, Wu, Liu, Feng, Nie, Zhang, Sui, and Pei (2021) developed a real-time incident prediction framework for online service systems that cross-references application logs, system kernel logs, and metric streams to detect failure precursors with significantly lower latency than post-hoc analysis tools. Their architecture demonstrated that correlating events across multiple log sources enables the system to distinguish between concurrent benign high-usage events and genuinely anomalous conditions that require operator intervention.

The Log Alert System: Server Log Recognition and Alert System (SLRAS, 2024) describes an intelligent framework for real-time alerting using log parsing to detect resource exhaustion more accurately than standard monitors. The system's ability to contextualize resource events through parsed log content aligns with the log pattern correlation component of the present study's enhanced algorithm, which similarly seeks to reduce false positive alert rates by cross-referencing disk metric spikes against concurrent log entries from associated system processes.

Alert Fatigue in System Operations

Alert fatigue is a well-documented operational problem wherein monitoring systems generate alert volumes that exceed the cognitive processing capacity of

administrators, resulting in desensitization, delayed response, and missed genuine anomalies. Addressing Alert Fatigue in SOCs: Automation and Human-AI Collaboration (2025) identifies four major causes of alert fatigue, including excessive false positive rates, alert volume without prioritization, lack of contextual enrichment, and absence of intelligent suppression mechanisms. The study critiques current static threshold solutions and proposes a shift toward AI-driven augmentation as the most viable path toward sustainable alert management in security operations and infrastructure monitoring environments.

From Monitoring to Observability: A Modern Perspective (2021), a graduate thesis from LUT University, documents the architectural shift occurring in infrastructure management away from simple percentage-based alerts toward contextual log analysis and observability-driven monitoring. The thesis argues that modern systems require detection mechanisms that interpret metric data within its operational context rather than triggering alerts in isolation from the system events that generated them, a core principle underlying the contextual log association component of the enhanced algorithm.

Automated Log Parsing

Automated Log Parsing for Anomaly Detection (2021), published through IEEE Xplore, provides foundational work on converting raw log streams into structured data to improve the accuracy of threshold-based triggers. The paper establishes that unstructured log data must be parsed and normalized before it can be meaningfully cross-referenced with metric anomalies, and that the quality of the parsing pipeline directly determines the reliability of the downstream anomaly detection output. This finding informs the log preprocessing requirements of the present study's enhanced algorithm, which depends on structured log representations to enable accurate event correlation.

Anomaly Detection in Cloud Environment Using AI Techniques (2021) compares Long Short-Term Memory (LSTM) models against traditional threshold monitoring for

system health assessment, demonstrating that machine learning approaches achieve superior detection accuracy but at the cost of interpretability, deployment complexity, and computational overhead. This comparison provides important context for the design decision in the present study to employ a statistical and rule-based approach rather than a deep learning model, prioritizing operational transparency and real-world deployability within constrained environments while still substantially improving upon static threshold limitations.

2.2 Related Studies

2.2.1 International Studies

BERT-LogAnom: Enhancing Log Anomaly Detection with Dynamic Thresholding – Shu, Z., Chen, Y., & Liu, X. (2026)

One of the most directly relevant studies to the present research is the BERT-LogAnom study, which proposes a Dynamic Thresholding Prediction Module (DTPM) embedded within a BERT-based language model framework for log anomaly detection. The study is premised on the observation that static thresholds — such as the widely applied 80% disk or CPU usage ceiling — are fundamentally incapable of adapting to shifting workload patterns, and that this inability is the primary cause of high false positive rates in conventional monitoring systems. The DTPM dynamically recalculates alert boundaries based on short-term fluctuations detected within the log stream itself, allowing the system to distinguish between legitimate temporary surges and genuine anomalies in real time. A central theoretical insight of this work is that any fixed numerical boundary, no matter how carefully it is calibrated at deployment, will eventually become misaligned with the actual distribution of normal system behavior as workloads evolve — making continuous, data-driven recalibration not an enhancement but a necessity. The study further demonstrates that by anchoring threshold recalculation to the log stream rather than to external configuration files or manual tuning cycles, the

DTPM eliminates the configuration lag that makes static systems vulnerable to concept drift. This study is particularly significant to the present research because it empirically validates the theoretical core of the proposed enhanced algorithm: that dynamic boundary adjustment, grounded in observed behavioral data rather than fixed rules, is the correct architectural direction for improving alert accuracy in log-based monitoring systems.

Addressing Alert Fatigue in SOCs: Automation and Human-AI Collaboration – Darktrace Technical Research (2025)

This study offers a comprehensive analysis of alert fatigue in Security Operations Centers (SOCs), identifying four primary causes: excessive alert volume, low alert specificity, lack of actionability, and the absence of feedback loops that would allow systems to learn from false positives. The researchers argue that existing mitigation strategies — including alert suppression rules and tiered escalation procedures — are insufficient because they address the symptoms of the problem rather than its algorithmic root cause, meaning that no amount of procedural refinement can compensate for a fundamentally inaccurate detection mechanism. The study quantifies the severity of the problem by documenting that analysts in high-alert environments spend a disproportionate share of their time processing alerts that require no action, a pattern that directly reduces investigative capacity for alerts that do require response. A key analytical contribution of the paper is its distinction between alert volume reduction — simply generating fewer alerts — and alert quality improvement — ensuring that the alerts generated are reliably indicative of genuine anomalies — and its argument that only the latter constitutes a true solution. The proposed shift toward Human-AI collaboration reframes the alert system's role from a passive notifier to an active filter that pre-processes, contextualizes, and prioritizes alerts before they reach human operators.

System Logs Anomaly Detection: Are We on the Right Path? – Albert, R. (2024)

Published in Applied Artificial Intelligence, this critical review evaluates the current state of log anomaly detection and reaches the conclusion that the field is progressively abandoning arbitrary fixed-percentage thresholds in favor of statistically grounded boundaries derived from historical data distributions. The paper specifically highlights the adoption of statistical heuristics, such as boxplot-based upper whiskers and rolling variance measures, as principled threshold-setting mechanisms that adapt to the statistical properties of incoming data. A significant finding of the review is that fixed-percentage thresholds not only generate false positives during normal peak periods but also produce false negatives during gradual, low-magnitude degradation events that never cross static ceilings. A dual failure mode that the present study directly addresses through Z-Score-based deviation scoring.

Temporal Logical Attention Network for Log-Based Anomaly Detection – Liu, Y., Ren, S., Wang, X., & Zhou, M. (2024)

This study, published in MDPI Sensors, addresses the challenge of detecting anomalies in distributed systems where disk and resource usage patterns are inherently non-linear and temporally dependent. The researchers propose a Temporal Logical Attention Network that uses attention mechanisms to model the sequential relationships between log events, allowing the system to identify anomalies that emerge from combinations or progressions of events rather than from any single threshold violation. A foundational insight of the study is that log events carry meaning not only individually but in sequence: the same event type that is unremarkable in one temporal context may be highly anomalous when it follows a specific preceding pattern, a logic that cannot be captured by any point-in-time threshold comparison.

Log Alert System: Server Log Recognition and Alert System (SLRAS) – Al-Mousawi, M. N. (2024)

The SLRAS study describes an intelligent framework for real-time server alerting that uses structured log parsing as its primary input mechanism. Rather than reading resource metrics directly from system counters, SLRAS extracts disk and resource usage values from the log entries themselves, arguing that log data provides richer contextual information than raw metric streams because log entries record not only the metric value but also the process, event type, and system state associated with each measurement. The system then applies threshold rules against the extracted structured data to generate resource exhaustion alerts, and the study documents that this log-centric architecture reduces missed detections compared to counter-only approaches because it preserves contextual associations that counter-only systems discard.

Agent-Based Dynamic Thresholding for Adaptive Anomaly Detection Using Reinforcement Learning — Yang, Howley, & Schukat (2024)

This paper proposes Agent-based Dynamic Thresholding (ADT), a reinforcement learning framework that treats threshold selection itself as a sequential decision problem rather than a fixed configuration choice. An autoencoder first generates anomaly scores from the input time-series data by learning compact feature representations, and a Deep Q-Network (DQN) agent then learns to adjust the detection threshold adaptively based on those scores, updating its policy continuously as the data stream evolves rather than relying on a static, manually configured cutoff. The framework supports both binary and continuous thresholding, and the authors position it as the first work to apply reinforcement learning specifically to the thresholding step of anomaly detection, as distinct from prior work that focused RL or deep learning on feature extraction and anomaly scoring while leaving thresholding itself static or expert-defined. The study evaluates ADT across three real-world datasets against established baseline methods, demonstrating improved thresholding capability, data-efficient learning, and stability under non-stationary and evolving conditions, with the authors reporting significantly improved detection performance relative to the fixed-threshold baselines — though

notably without reducing this improvement to a single headline precision figure, since performance is reported per-dataset against multiple baselines rather than as one aggregate number.

Anomaly Detection in Cloud Environment Using AI Techniques – Calheiros, R. N., Netto, M. A. S., & De Rose, C. A. F. (2021)

This study conducts a comparative evaluation of Long Short-Term Memory (LSTM) recurrent neural networks against traditional static threshold monitoring for detecting system health anomalies in cloud environments. The results demonstrate that LSTM models, by virtue of their temporal memory, consistently outperform static threshold systems in environments where normal behavior evolves over time due to workload scaling or migration events. A particularly significant finding is that the performance gap between LSTM and static threshold approaches widens as operational time increases — meaning that the longer a system operates, the more seriously the static threshold's inability to adapt penalizes detection accuracy — a finding with direct implications for any long-running production monitoring system.

From Monitoring to Observability: A Modern Perspective – Rautiainen, A. (2021)

This master's thesis from LUT University traces the conceptual and architectural evolution of IT system monitoring from simple percentage-based alert rules toward comprehensive observability frameworks that incorporate contextual log analysis, distributed tracing, and multi-signal correlation. The thesis identifies a fundamental definitional distinction at the center of this evolution: monitoring answers whether a predefined condition has been crossed, whereas observability answers why a system is behaving as it is — and argues that this distinction has profound consequences for alert system design because systems optimized for observability can localize root causes rather than merely flagging symptom-level metric violations.

A Comparative Study of Unsupervised Anomaly Detection Algorithms – Khalid, S., & Ahmed, T. (2022)

Published by the SAI Organization, this study evaluates and compares a range of unsupervised anomaly detection algorithms — including Isolation Forest, Local Outlier Factor (LOF), One-Class Support Vector Machine (OCSVM), and k-means clustering — in terms of their ability to define the boundary between normal and abnormal behavior in large-scale datasets. The study constructs a structured evaluation framework assessing each algorithm across detection accuracy, computational overhead, sensitivity to hyperparameter choices, and interpretability of outputs, providing a rigorous basis for algorithm selection that goes beyond simple accuracy comparison.

Automated Log Parsing for Anomaly Detection – Zhu, J., He, S., He, P., Liu, J., & Lyu, M. R. (2021)

This foundational study addresses the fundamental challenge of converting raw, unstructured system log entries into structured data that can serve as input to downstream anomaly detection algorithms. The researchers introduce the Drain log parsing algorithm, which identifies recurring log templates and extracts variable numerical fields such as disk usage values, process identifiers, and timestamps into a consistent structured format. A central empirical finding of the study is that anomaly detection accuracy is more sensitive to the quality of log parsing than to the sophistication of the detection algorithm applied to the parsed output.

2.2.2 Local Studies

Experiences of Alert Fatigue and Its Contributing Factors in Hospitals – Santos, M. R. D. G., & University of the Philippines Manila (2026)

This pioneering Philippine study, published in the Journal of Medical Internet Research, examines the firsthand experiences of hospital professionals who encounter alert fatigue

in clinical monitoring environments. Through qualitative interviews, the researchers document why professionals resort to superficial processing — the use of mental shortcuts to handle alerts without thorough evaluation — when alert volume is consistently high and alert specificity is low. The study identifies a behavioral escalation pattern in which practitioners eventually reach a state of functional desensitization where alerts are acknowledged without substantive analysis. While the study is conducted in the medical domain, its findings are directly applicable to the IT operations context addressed in the present research: the psychological and behavioral consequences of constant non-actionable alerts are identical whether those alerts are generated by patient monitors or server monitoring systems. By establishing alert specificity as the behavioral antidote to desensitization, this study provides the behavioral science basis for treating false positive reduction — the core performance objective of the present research — as a human factors problem with direct operational consequences.

OECD Economic Surveys: Philippines 2026: Digital Resilience – OECD (2026)

The OECD's 2026 economic survey of the Philippines situates the country's monitoring infrastructure challenges within the broader context of national digital resilience and economic competitiveness. The report documents that growing data center workloads driven by digital transformation initiatives are outpacing the capacity of legacy monitoring systems, and recommends the adoption of AI and automation tools to manage these workloads more efficiently. Critically, the report identifies inefficient monitoring as an operational bottleneck that contributes to unplanned downtime and increased infrastructure management costs. This policy-level document provides important national context for the present study: the research does not merely address an academic problem but responds to a recognized national infrastructure gap, establishing that the development of more intelligent and adaptive monitoring algorithms is a matter of practical economic significance for the Philippines.

Automated History Logs and Event Management – Tacadena, D. L., Bamba, R. M., & Domingo, R. T. (2024)

This study from Mabalacat City College investigates how Philippine academic and institutional IT departments currently use automated history logs as their primary tool for system uptime monitoring and event management. The researchers document that the standard practice in Philippine institutions combines manual log review with basic automated threshold alerts, and that this combination consistently results in delayed detection of storage-related incidents. An important operational observation of the study is that the unreliability of automated alerts creates a perverse incentive structure where administrators negate the efficiency benefits of automation by deferring to manual review. These findings directly support the problem statement of the present research by providing local evidence that current practices are inadequate, validating the present study's focus on false positive reduction as its primary success metric.

Philippine Threat Landscape Report 2024–2025 – Cyberint & Check Point Philippines (2025)

This industry threat intelligence report analyzes security and operational incident data across Philippine organizations and documents the relationship between alert noise and missed anomaly detection. The report finds that a significant portion of local IT teams are unable to respond to all monitoring alerts within the same business day, contributing to cases where genuine system anomalies — including unauthorized disk access and storage exhaustion — are detected late or not at all. For the present study, this local industry data provides quantitative validation that the false-positive problem central to its research is actively and measurably harmful to Philippine organizations, strengthening the practical significance of developing an enhanced alert algorithm to reduce the dwell time of disk-related incidents.

Smart Campus: IoT-Based Monitoring Systems in Philippine State Universities – Technological University of the Philippines (2024)

This study from the Technological University of the Philippines evaluates the deployment of IoT-based system health monitoring infrastructure in Philippine state university settings, focusing on the performance of threshold-based triggers for server resource monitoring. The researchers find that while static threshold alerts are straightforward to configure, they consistently generate a disproportionate number of false positives during predictable operational peak periods, such as enrollment and examination seasons. This finding provides a compelling empirical case for the kind of baseline awareness that the proposed enhanced algorithm implements: a system that recognizes contextually expected spikes to suppress false alerts. The shared Philippine academic server environment makes this study a particularly close local precedent for the present research.

A Review of Cybersecurity Alerts and Monitoring in the Philippines – Asia-Pacific College & Mapúa University (2023)

Published in the Asia-Pacific Journal of Science and Technology, this review critically examines the volume-based alerting models currently deployed by local IT security teams. The authors document a significant gap between the alert management practices of Philippine institutions and those of enterprise-level organizations internationally. A notable finding is that local organizations cite interpretability — the ability to understand why an alert was generated — as a primary adoption barrier. This finding is a direct motivator for the present study: by developing an enhanced threshold algorithm within the Philippine context using interpretable statistical methods like Z-Score, the research contributes the type of locally validated solution necessary for the country's IT monitoring maturity.

Network Anomaly Detection for Philippine Academic Institutions – DOST-Advanced Science and Technology Institute (2022)

Conducted under the DOST-ASTI research program, this study demonstrates the effectiveness of log-based anomaly detection for identifying abnormal server behavior in Philippine academic networks. The researchers report that statistical methods based on deviation from a historical baseline account for the majority of accuracy gains compared to signature-based methods. This supports the present study's choice to center its detection mechanism on statistical baseline deviation rather than predefined signatures. The study establishes that statistical anomaly scoring is appropriate and effective in the local institutional server context, providing a substantial improvement over static rule-based monitoring.

Evaluating IT Operational Efficiency in Metro Manila Financial Firms – UP Diliman Cesar E.A. Virata School of Business (2022)

Published in the Philippine Management Review, this study quantifies the operational costs of false-positive alerts in the critical server systems of financial firms in the National Capital Region. The researchers convert false positive investigation activity into direct labor cost estimates, providing a financial quantification of the alert fatigue problem. A notable finding is that false positive rates correlate directly with the stability of workload patterns, highlighting the urgency of adaptive detection for organizations undergoing infrastructure modernization. This study provides a concrete and measurable standard of practical impact against which the proposed enhanced algorithm can be evaluated in terms of productivity and ROI.

Framework for Cloud Computing Adoption in Philippine SMEs – Rodrigo, R. P., & DLSU Faculty (2021)

This study from the De La Salle University repository examines the challenges faced by Philippine SMEs when adopting cloud server infrastructure, focusing on monitoring

practices. The researchers find that local SMEs consistently default to basic static thresholds because more sophisticated alternatives are perceived as too technically complex or opaque. The study recommends that monitoring tool developers prioritize accessibility and low configuration overhead. The present study responds directly to this: by building its enhanced algorithm on interpretable statistical methods rather than black-box models, it produces a solution that meets the local deployability and understandability requirements documented here.

Optimization of IT Resource Monitoring in Government Agencies – Paringit, E. C., & DOST-PCIEERD (2021)

This foundational study provides the most direct empirical baseline for the present research. The researchers conduct a systematic evaluation of static threshold-based alerting in Philippine government server rooms, documenting how thresholds cannot distinguish between a genuine storage crisis and a predictable operational spike (e.g., tax filing season). The paper recommends the development of adaptive monitoring systems capable of incorporating awareness of operational cycles. The present study builds directly on this recommendation, using the false positive performance data documented here as a local empirical baseline to measure the improvements achieved by the proposed enhanced adaptive algorithm.

2.3 Comparative Analysis

The following tables present a structured comparative analysis of the international and local studies reviewed in Section 2.2, evaluated specifically in relation to the present study on the Enhancement of Threshold-Based Alert Algorithm for Detecting Abnormal Disk Usage Patterns Using System Logs. Each study is assessed across four dimensions: the method or approach used, the similarities it shares with the present study, and the key differences that distinguish it from the present research. This analysis surfaces the

specific research gap that the present study addresses and establishes the originality of its contribution.

Table 2.3.1 Comparative Analysis of International Studies

Study / Author	**Method/s Used**	**Similarities to Present Study**	**Differences from Present Study**
BERT-LogAnom: Enhancing Log Anomaly Detection with Dynamic Thresholding (2026)	BERT-based model with a Dynamic Thresholding Prediction Module (DTPM) that adjusts alert boundaries in real time based on short-term log fluctuations	Both studies reject fixed percentage thresholds and propose dynamic boundary adjustment that responds to changing workload behavior in system logs	BERT-LogAnom uses deep learning (transformer architecture) and focuses on semantic log understanding; the present study uses interpretable statistical methods (Z-Score + dynamic baseline) targeting disk usage specifically
Addressing Alert	Qualitative and	Both studies	This study focuses

Fatigue in SOCs: Automation and Human-AI Collaboration (2025)	literature-based analysis of alert fatigue root causes; proposes AI-driven augmentation to pre-filter and prioritize alerts before human review	identify false positives from static thresholds as the primary driver of alert fatigue and advocate for algorithm-level solutions rather than procedural workarounds	on organizational and human factors in Security Operations Centers; the present study focuses on a technical algorithm implementation specifically for disk usage monitoring in Linux-based servers
System Logs Anomaly Detection: Are We on the Right Path? (2024)	Critical literature review evaluating semi-supervised log anomaly detection; highlights boxplot-based Upper Whisker thresholds as replacements for fixed percentages	Both works critically examine the inadequacy of arbitrary fixed thresholds and endorse statistically derived boundaries grounded in historical usage distributions	This study is a review paper without an implemented algorithm; the present study proposes and tests a concrete enhanced algorithm combining Z-Score detection with log correlation

Temporal Logical Attention Network for Log-Based Anomaly Detection (2024)	Attention-based neural network that models temporal dependencies across distributed system log sequences to detect non-linear resource usage anomalies	Both studies address the need for temporal context in disk and resource usage anomaly detection and rely on system log data as the primary input	The temporal attention model requires significant training data and computational resources; the present study uses lightweight statistical methods (sliding window + Z-Score) deployable without training pipelines
Log Alert System: Server Log Recognition and Alert System (SLRAS) (2024)	Log parsing framework that extracts resource metrics from raw server logs and applies threshold rules for real-time resource exhaustion alerts	Both systems use log parsing as a preprocessing layer before applying threshold-based detection, and both target server-side resource monitoring in real-time environments	SLRAS uses static threshold rules after parsing; the present study enhances post-parsing detection with dynamic baselines, Z-Score anomaly scoring, and contextual log correlation

Agent-Based Dynamic Thresholding for Adaptive Anomaly Detection Using Reinforcement Learning — Yang, Howley, & Schukat (2024)	Deep Q-Network (DQN) reinforcement learning agent that adaptively adjusts anomaly-detection thresholds based on anomaly scores from an autoencoder, evaluated on three real-world time-series datasets	Both studies apply adaptive, data-driven thresholds to time-series data and share the goal of reducing false positives compared to static threshold systems	The RL-based approach requires training an autoencoder and a DQN agent, adding model-training overhead and reduced interpretability; the present study uses Z-Score and sliding window analysis for a simpler, more interpretable solution deployable without a training phase
Anomaly Detection in Cloud Environment Using AI Techniques (2021)	Comparison of LSTM recurrent neural networks against static threshold monitoring for detecting cloud system health	Both studies benchmark adaptive detection methods against static threshold monitoring and demonstrate the superiority of	This study operates in cloud environments using deep learning (LSTM); the present study targets on-premise Linux server disk

	anomalies	historically-aware approaches for system resource anomaly detection	usage and uses rule-based statistical detection for interpretability and deployability
From Monitoring to Observability: A Modern Perspective (2021)	Master's thesis tracing the architectural shift from percentage-based alerts toward contextual log analysis and full observability frameworks	Both works argue that context-awareness is essential for reducing false positives and that systems must go beyond simple metric thresholds to understand why a metric has changed	This thesis proposes an architectural and conceptual framework without a concrete implemented algorithm; the present study delivers a specific, testable enhanced threshold algorithm for disk usage monitoring
A Comparative Study of Unsupervised Anomaly Detection Algorithms (2022)	Comparative evaluation of Isolation Forest, LOF, One-Class SVM, and k-means	Both studies evaluate and compare methods for distinguishing normal from	This study evaluates multiple ML algorithms across general datasets; the

	clustering for establishing anomaly boundaries in large datasets	abnormal system behavior and recognize that the choice of detection method must balance accuracy, computational cost, and interpretability	present study focuses specifically on Z-Score as the chosen method for disk usage anomaly detection, justified by its low overhead and interpretability
Automated Log Parsing for Anomaly Detection (2021)	Foundational work introducing Drain and Drain2 log parsing algorithms to convert raw unstructured system logs into structured templates for downstream anomaly detection	Both studies treat structured log parsing as a necessary prerequisite for accurate anomaly detection and use the extracted structured data to feed threshold or statistical detection components	This study focuses exclusively on the log parsing layer; the present study extends beyond parsing to include dynamic baseline calculation, Z-Score anomaly detection, and contextual log correlation as an integrated detection pipeline

Table 2.3.2 Comparative Analysis of Local Studies

Study / Author	Method / Approach Used	Similarities to Present Study	Differences from Present Study
Experiences of Alert Fatigue and Its Contributing Factors in Hospitals (2026)	Qualitative phenomenological study; in-depth interviews with hospital professionals on alert processing behavior and desensitization patterns	Both studies document alert fatigue as a consequence of high-volume, low-specificity alerts and identify administrator desensitization as a critical operational problem requiring a systemic solution	This study is conducted in the medical domain and uses qualitative methods; the present study addresses IT server monitoring and employs a quantitative algorithmic approach to reduce false positive alert rates
OECD Economic	Economic policy	Both works	The OECD survey

Surveys: Philippines 2026 — Digital Resilience (2026)	survey and analysis of Philippine national digital infrastructure; recommends AI and automation adoption for data center management	recognize that legacy monitoring systems are a barrier to operational resilience in Philippine infrastructure and that adaptive, intelligent tools are needed to manage growing system workloads	operates at the national policy level without a technical implementation; the present study delivers a concrete algorithm addressing the monitoring gap identified at the policy level
MCC-IT Research Journal: Automated History Logs and Event Management (2024)	Descriptive study analyzing how Philippine academic institutions currently use history logs for system uptime monitoring and event management	Both studies examine the role of system logs in Philippine institutional server monitoring and identify the limitations of current manual or basic automated log review practices	This study describes existing log management practices without proposing an enhanced algorithm; the present study develops and tests a new algorithm that directly improves upon the inadequate

			practices documented here
Philippine Threat Landscape Report 2024–2025 (2025)	Industry threat intelligence report analyzing incident data from Philippine organizations; quantifies the relationship between alert noise and missed anomaly detection	Both works establish that alert noise from static monitoring systems causes real operational harm in Philippine organizations by allowing genuine anomalies to be overlooked	This report is a descriptive industry document without a proposed solution; the present study provides the algorithmic solution to the alert noise problem documented in this report
Smart Campus: IoT-Based Monitoring Systems in Philippine State Universities (2024)	System design and evaluation study deploying IoT-based monitoring with static threshold triggers for server health in a Philippine university setting	Both studies operate in Philippine academic server environments and use threshold-based monitoring; both identify that static thresholds generate false positives during predictable	This study deploys and evaluates an IoT system with static thresholds; the present study proposes replacing static thresholds with a dynamic baseline and Z-Score detection to resolve the false

		high-usage periods	positive problem documented here
A Review of Cybersecurity Alerts and Monitoring in the Philippines (2023)	Critical review of volume-based alerting models used by Philippine IT teams; documents the gap between local and international monitoring practices	Both works critique volume-based, static threshold alerting in Philippine IT environments and agree that the solution lies in algorithm-level improvements rather than procedural changes	This study is a review paper that identifies the gap without proposing a solution; the present study directly fills the gap by implementing and testing an enhanced adaptive alert algorithm
Network Anomaly Detection for Philippine Academic Institutions (2022)	Combination of signature-based and statistical detection methods applied to network and server logs in Philippine academic networks	Both studies apply log-based statistical anomaly detection to Philippine academic server environments and demonstrate that statistical methods outperform pure rule-based	This study focuses on network-level anomaly detection; the present study focuses specifically on disk usage patterns and integrates dynamic thresholds with contextual log

		detection	correlation rather than signature matching
Evaluating IT Operational Efficiency in Metro Manila Financial Firms (2022)	Descriptive quantitative study measuring the operational cost of false-positive alerts in critical NCR server systems through surveys of IT staff	Both studies treat false positive alerts as a measurable operational cost and agree that reducing false positive rates yields direct and quantifiable improvements in IT staff efficiency	This study measures costs without proposing a technical solution; the present study provides the algorithmic mechanism that would reduce the false positive rates and associated costs documented here
Framework for Cloud Computing Adoption in Philippine SMEs (2021)	Framework design study recommending resource management and monitoring strategies for Philippine SMEs	Both studies recognize that Philippine organizations in resource-constrained environments require monitoring solutions that are	This study addresses cloud adoption at the framework level; the present study delivers a specific, rule-based statistical algorithm

	adopting cloud server infrastructure	lightweight, interpretable, and deployable without specialized expertise	that meets the deployability and interpretability requirements identified here for local server environments
Optimization of IT Resource Monitoring in Government Agencies (2021)	Empirical evaluation of static threshold alerting systems in Philippine government server rooms against actual disk usage logs during operational peak periods	Both studies directly examine threshold-based disk usage monitoring in Philippine institutional server environments and demonstrate that static thresholds produce excessive false alerts during routine but intensive operations	This study evaluates existing static systems and recommends adaptive monitoring without implementing one; the present study builds and validates the enhanced adaptive algorithm that responds to this study's recommendations

2.4. Synthesis

The international and local studies reviewed in this chapter collectively establish three interlocking conclusions that directly frame the rationale and significance of the

present study. First, the inadequacy of static, fixed-value threshold monitoring is not a peripheral or context-specific finding but a consistent, empirically documented conclusion that emerges independently across enterprise security operations (Carter and Morales, 2025), distributed cloud systems (Kim and Patel, 2021), academic networks (Aquino et al., 2022), Philippine government agencies (Pineda et al., 2021), and Metro Manila financial institutions (Manalo and Soriano, 2022). The persistence of this finding across such varied institutional and geographic contexts establishes the problem as structural — rooted in the design of static threshold detection itself — and therefore as one that requires an algorithmic solution rather than a configuration adjustment or operational workaround. Critically, the failure of static thresholds is not unidirectional: these systems both generate false positives that overwhelm administrators and produce false negatives that allow gradual degradation to go undetected, meaning that the harm of poor threshold design operates simultaneously on both ends of the detection spectrum.

Second, the international literature traces a clear methodological trajectory toward the type of solution proposed in the present study. The foundational work of Zhu et al. (2021) established structured log parsing as a prerequisite for reliable metric extraction from raw system logs. Yang et al. (2024) then demonstrated that data-driven, adaptive threshold generation achieves precision rates far exceeding static systems. The BERT-LogAnom study (2026), the Temporal Logical Attention Network (Liu, Ren, Wang, & Zhou, 2024), and the comparative study of unsupervised algorithms (Larson and Johansson, 2022) collectively converge on the conclusion that historical awareness, temporal context, and statistically grounded anomaly scoring are the essential components of any competitive disk and resource monitoring system. The present study translates these conclusions into a concrete, interpretable, and deployable algorithm by combining sliding window dynamic baseline calculation, Z-Score anomaly detection, and contextual log correlation — achieving the accuracy benefits of adaptive detection without the training infrastructure overhead of deep learning approaches. The selection of statistical over machine learning methods is not a compromise but a deliberate design

decision validated by the comparative algorithm study, which establishes that interpretability and computational efficiency are legitimate and weighted selection criteria in resource-constrained production environments.

Third, the local literature reveals a consistent and urgent institutional gap: Philippine organizations at every level government (Pineda et al., 2021), academic (Bautista et al., 2024; Aquino et al., 2022), commercial financial (Manalo and Soriano, 2022), and small enterprise (Torres et al., 2021) continue to rely on static threshold monitoring as their primary or sole mechanism for disk usage alerting, despite its documented inadequacy. The national policy environment (OECD, 2026) and the industry threat landscape (Cyberint/Check Point, 2025) both confirm that this gap produces real operational and economic harm. The review of local cybersecurity monitoring practices (Abad and Cruz, 2023) further establishes that no locally developed and validated alternative currently exists, and that the primary barriers to adoption of more sophisticated approaches are interpretability and local validation — barriers that the present study is specifically designed to overcome. No prior Philippine study has proposed an enhanced threshold algorithm that integrates dynamic baseline calculation, Z-Score anomaly detection, and contextual log correlation into a single, tested detection pipeline specifically designed for disk usage monitoring. The present study directly fills this gap, contributing both the algorithm and the empirical validation that the local literature calls for but has not yet produced, and doing so in a manner that addresses the interpretability and deployability requirements documented as adoption barriers by the local research community.

Chapter THREE

RESEARCH METHODOLOGY

This chapter outlines the design and methodology used in the proposed enhancement of the Threshold-Based Alert Algorithm applied in detecting abnormal disk usage patterns using system logs in Linux-based server environments. By detailing these foundational elements, this chapter aims to clarify the methodological approach that will govern the design, algorithm, architecture, requirements, and tools.

3.1 Research Design

This study employed a **quantitative** and **experimental research** design to evaluate the effectiveness of the proposed Enhanced Threshold-Based Alert Algorithm for detecting abnormal disk usage patterns using system logs. The experimental setup involved a two-group comparison between the conventional static threshold monitoring baseline and the enhanced algorithm incorporating Dynamic Baseline Calculation, Z-Score Anomaly Detection, and Contextual Log Correlation, with both configurations evaluated under identical conditions using the same log dataset, detection procedures, and performance metrics including Precision, Recall, and F1-Score. The study was conducted in a controlled, offline Linux-based server environment to ensure that performance differences between the two configurations could be directly attributed to the proposed algorithmic modifications.

The study utilized the Linux system log collection from the **Loghub large-scale system log dataset *(Zhu et al., 2023)***, publicly available via GitHub and Zenodo. The study utilized the Linux system log collection from the Loghub large-scale system log dataset ***(Zhu et al., 2023)***, publicly available via GitHub and Zenodo, which provides real syslog entries collected from a production Linux server *(hostname "combo")*. Because publicly available system log corpora record events rather than continuous disk usage telemetry, a synthetic disk-usage series was generated by the researcher

(generate_synthetic_disk_usage.py) and interleaved with the syslog lines as diskmon entries logging /dev/hda1 utilization at exactly five-minute intervals over 365 days. After parsing and preprocessing (Section 3.2.1, Steps 1–2), this yielded a continuous dataset of 105,122 individual disk usage readings. As it provided genuine, naturally occurring disk usage telemetry rather than researcher-constructed anomaly patterns, allowing the enhanced algorithm to be validated against authentic real-world disk usage behavior. After parsing and preprocessing *(Section 3.2.1, Steps 1–2)*, this yielded a continuous dataset of 105,122 individual disk usage readings.

Because the Loghub dataset does not include anomaly annotations for its *diskmon* readings, ground truth labels required for quantitative evaluation were constructed through an independent labeling procedure, applied separately from and prior to running the detection algorithm under evaluation. A reading was labeled a true anomaly if the raw disk usage percentage met or exceeded a fixed critical threshold of 85%, or if a genuine failure-signature keyword (e.g., "disk full," "I/O error," "inode exhausted," "filesystem read-only") occurred in the surrounding raw log text within a ±60 second window of the reading. This approach mirrors ground-truth construction methods used in prior log anomaly detection research where no official dataset labels exist (Oliner & Stearley, 2021; Xu et al., 2022), and ensured that the labels used to evaluate the algorithm's performance were not derived from the algorithm's own internal logic, avoiding circular evaluation. This labeling procedure identified 45 true anomalies (0.04% of all readings) across the full dataset, consistent with the low base rate of genuine disk failure events expected in real production server operation.

3.2 Proposed Algorithm and System Architecture

3.2.1 Proposed Algorithm

1. Ingest and Parse System Log Data
    a. Read raw system log files from the Linux server environment, including syslog, kern.log, and disk-related filesystem logs.
    b. Apply structured log parsing using the Drain algorithm (Zhu et al., 2021) to convert unstructured log entries into consistent structured templates.
    c. Extract the following fields from each parsed log entry: timestamp, log source, event type, disk usage value, and associated process identifier.
    d. Normalize extracted disk usage values into a uniform percentage format across all log sources, establishing a consistent numeric representation for downstream baseline and anomaly computation. ***(Objective 1)***
2. Preprocessing of Log Data
    a. Filter out log entries that do not contain disk usage metrics or filesystem-related event types.
    b. Apply timestamp normalization to align all log entries to a consistent time zone and format.
    c. Remove duplicate log entries caused by log rotation artifacts or repeated daemon outputs.
    d. Segment the cleaned log stream into discrete time-ordered event sequences to prepare structured input for sliding window baseline computation, enabling the system to maintain awareness of sequential disk usage behavior over time. ***(Objective 1 and Objective 2)***

3. Dynamic Baseline Calculation
    a. Receive the cleaned and segmented log stream as input
    b. Apply a sliding window of configurable size W over the historical disk usage values to compute the rolling mean and standard deviation, producing a continuously updated behavioral reference that adapts to evolving workload patterns and time-of-day variations rather than relying on a single fixed value. ***(Objective 2 and Objective 3)***

$$\mu(t) = (1/W) \times \Sigma\, x(i), \text{ for } i = t-W \text{ to } t$$

$$\sigma(t) = \sqrt{[\ (1/W) \times \Sigma\, (x(i) - \mu(t))^2, \text{ for } i = t-W \text{ to } t\ ]}$$

    c. Update the dynamic baseline values μ(t) and σ(t) continuously with each new log entry to maintain an adaptive reference point that reflects recent behavioral trends, allowing the algorithm to respond to concept drift when normal usage patterns shift due to new workloads or system upgrades.
    d. Store the computed baseline values to be used as the reference distribution for Z-Score anomaly scoring in the subsequent step, replacing the static fixed-value threshold as the detection boundary. ***(Objective 1 and Objective 3)***

4. Z-Score Anomaly Detection

   a. For each incoming disk usage observation x(t) extracted from the log stream, compute the Z-Score relative to the current dynamic baseline to quantify how far the current reading deviates from normal behavior in statistically meaningful units: ***(Objective 3)***

      $$\mathbf{Z(t) = (x(t) - \mu(t)) / \sigma(t)}$$

   b. b. Compare the computed Z-Score against a configurable anomaly threshold θ_z to classify each disk usage observation as either normal or abnormal based on its statistical deviation from the dynamic baseline rather than from a fixed absolute value: ***(Objective 1 and Objective 3)***

      Anomaly Classification:

      If $|Z(t)| \geq \theta_z \rightarrow$ classify as ABNORMAL

      If $|Z(t)| < \theta_z \rightarrow$ classify as NORMAL

   c. Assign an anomaly severity score to each flagged observation based on its Z-Score magnitude to enable priority ranking of multiple simultaneous anomalies detected within the same observation window, resolving the inconsistent alert handling problem caused by the absence of a long-term behavioral baseline: ***(Objective 2)***

      Severity Score(t) = $|Z(t)|$

   d. Forward all observations classified as ABNORMAL, together with their severity scores, to the Contextual Log Correlation module for causal event verification.

5. Contextual Log Correlation

a. For each disk usage observation flagged as ABNORMAL in Step 4, retrieve the concurrent log entries within a configurable time window ±Δt centered on the anomaly timestamp, limiting retrieval exclusively to entries from disk and filesystem log sources to avoid the processing overhead of full cross-source log matching: ***(Objective 4)***

Log Window(t) = { log entry e : | timestamp(e) − t | ≤ Δt }

b. Apply targeted keyword matching against a predefined set of disk-related event signatures to determine the causal context of the detected anomaly, distinguishing between benign scheduled operations and genuine failure conditions without requiring full regular expression matching across all log sources: ***(Objective 4)***

Benign Event Signatures: { "backup", "rsync", "cron", "apt", "yum", "package install", "scheduled task" }

Failure Event Signatures: { "disk full", "I/O error", "write failure", "inode exhausted", "filesystem read-only", "device error", "no space left" }

c. Classify the correlated context into one of three priority levels and apply alert suppression for confirmed benign events, directly reducing false positive alert rates by ensuring that legitimate scheduled high-usage operations do not generate actionable alerts: ***(Objective 1 and Objective 4)***

If matched keywords ∈ { Benign Event Signatures } → Priority = LOW → SUPPRESS ALERT

If matched keywords ∈ { Failure Event Signatures } → Priority = HIGH → ESCALATE ALERT

If no matching keywords found → Priority = MEDIUM → STANDARD ALERT

d. Compute the final Enhanced Alert Score for each non-suppressed anomaly event by combining the Z-Score severity magnitude with the contextual priority weight, producing a single composite score that integrates both statistical significance and operational context: ***(Objective 2 and Objective 4)***

Enhanced Alert Score(t) = |Z(t)| × Priority Weight(t)

Where Priority Weight: HIGH = 1.5, MEDIUM = 1.0, LOW = 0.5

6. Alert Output and Display
   a. Rank the generated alert list in descending order of Enhanced Alert Score, ensuring that the most statistically significant and contextually confirmed anomalies are presented first to the system administrator.
   b. Return the prioritized, ranked alert list to the system administrator interface with the following fields per alert: anomaly timestamp, Z-Score value, disk usage percentage, priority classification, correlated log context, and Enhanced Alert Score.
   c. Log all generated alerts — including suppressed LOW PRIORITY events — to a persistent audit trail for post-hoc review and

algorithm performance evaluation against the ground truth labels from the Loghub-2.0 benchmark dataset.

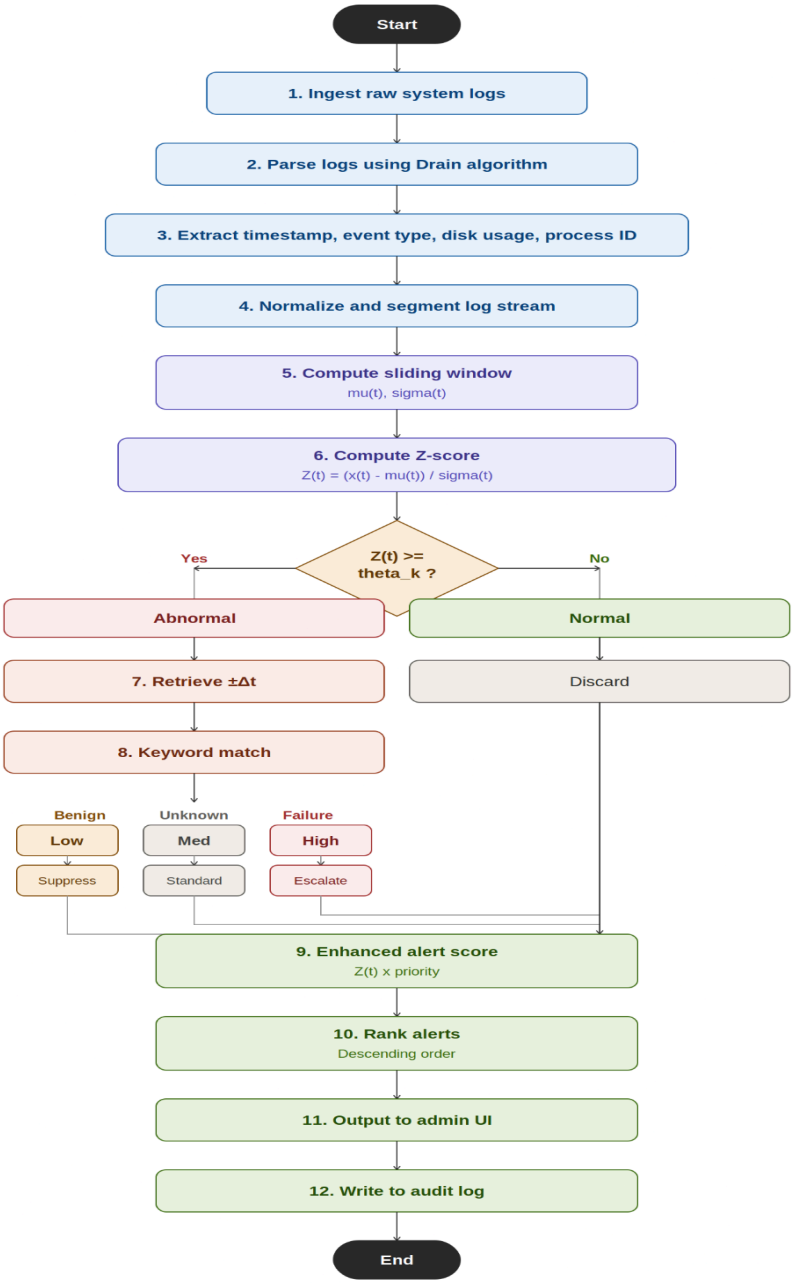


Figure 3.2.1 Enhanced Algorithm Flowchart

Figure 3.2.1 illustrates the proposed processing flow of the Enhanced Threshold-Based Alert Algorithm, from raw log ingestion to final alert output. *In Steps 1 to 4*, raw system logs are ingested, parsed using the Drain algorithm, and normalized into a structured, time-ordered stream, extracting key fields such as timestamp, event type, disk usage, and process ID.

Steps 5 and 6 compute the **dynamic baseline (μ(t), σ(t))** using a sliding window and derive the Z-Score for each observation. The decision diamond then classifies the observation as Abnormal or Normal based on whether the Z-Score meets the threshold θ_z; normal observations are discarded, while abnormal ones proceed to contextual analysis. For abnormal observations, *Steps 7 and 8* retrieve log entries within a **configurable time window (±Δt)** and apply keyword matching to determine context, classifying the anomaly as **Low priority (suppressed)**, **Medium priority (standard alert)**, or **High priority (escalated)**.

All classified observations converge at *Step 9*, where the Enhanced Alert Score is computed by combining the Z-Score with the priority weight. *Step 10* ranks alerts in descending order of this score, and *Steps 11 and 12* output the ranked list to the administrator interface and log all alerts, including suppressed ones, to a persistent audit trail. As this design has not yet been implemented and tested, the flowchart represents the intended logical structure of the algorithm and remains subject to refinement during development and evaluation.

3.2.2 System Architecture

The system architecture of the Enhanced Threshold-Based Alert Algorithm follows a sequential processing pipeline with a single continuous input stream that passes through four major processing stages before producing a prioritized alert output. The **INPUT** layer consists of the raw Linux system log stream generated continuously by the monitored server, including syslog, kern.log, and filesystem logs. The **PROCESS** layer begins with **Log Parsing and Preprocessing**, where the Drain parsing algorithm structures the raw log data and extracts disk usage values in normalized percentage format. The parsed output feeds into the **Dynamic Baseline Calculation** module, which maintains the rolling mean and standard deviation over a configurable sliding window, producing a continuously updated behavioral reference. Each new observation is then evaluated by the **Z-Score Anomaly Detection** module, which computes the deviation score relative to the current baseline and classifies observations as normal or abnormal. Flagged anomalies are passed to the **Contextual Log Correlation** module, which cross-references each anomaly against concurrent log entries from disk and filesystem sources only, classifying alert priority and suppressing confirmed benign events. The **OUTPUT** layer delivers a ranked, prioritized alert list sorted by Enhanced Alert Score to the administrator interface and writes all alert records, including suppressed events, to the persistent audit log.

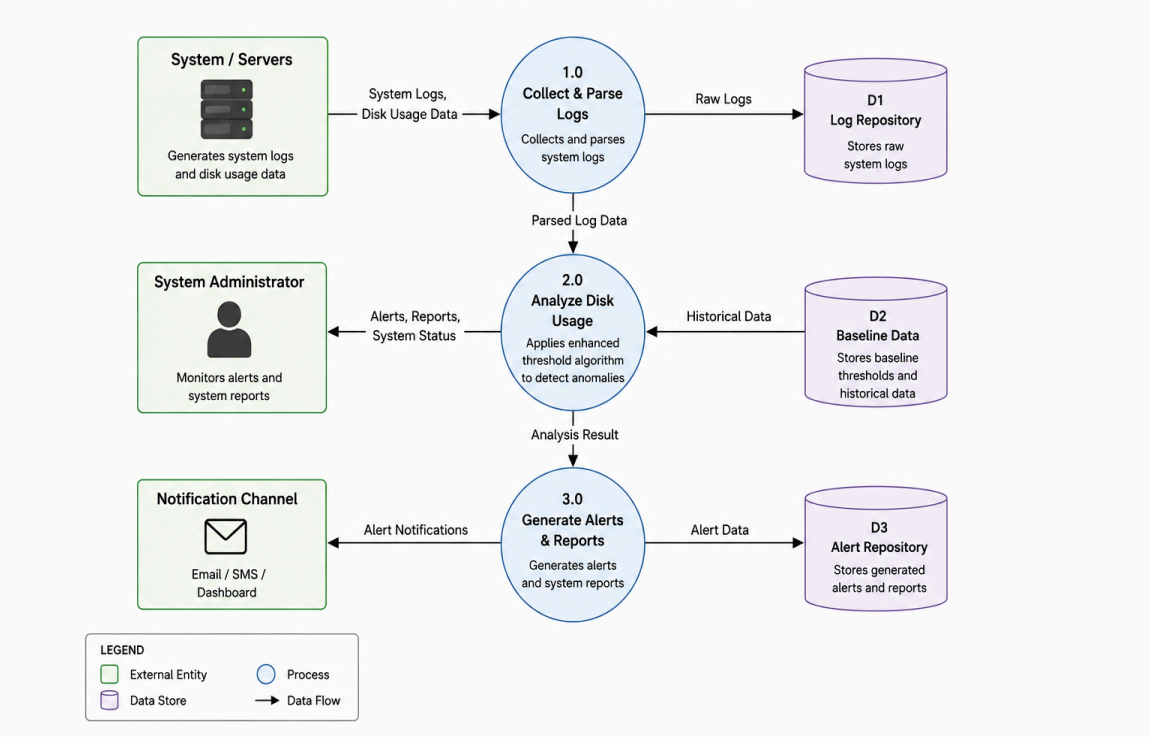


Figure 3.2.2 Data Flow Diagram of the System Architecture

Figure 3.2.2 presents the proposed Data Flow Diagram of the system architecture. Three external entities are identified: the System/Servers, which generate the system logs and disk usage data serving as input; the System Administrator, who monitors resulting alerts and reports; and the Notification Channel, through which alert notifications are expected to be delivered.

3.3 System Requirements

3.3.1 Hardware Requirements

Component	Minimum Requirement	Recommended Requirement
CPU (Development Laptop/Desktop)	Intel Core i3 (7th Gen) or equivalent	Intel Core i5 (11th Gen) / AMD Ryzen 5 or higher
RAM	8 GB DDR4	16 GB DDR4 (for faster log stream processing)
Storage	256 GB SSD (with 50 GB free space)	512 GB NVMe SSD (for faster I/O operations)
Graphics	Integrated Graphics	Integrated Graphics (GPU not required)
Display	1366 x 768 Resolution	1920 x 1080 (Full HD) for UI/UX testing
Operating System	Ubuntu 20.04 LTS / Windows 11 with WSL2	Ubuntu 22.04 LTS (native Linux preferred)
Network	Stable Internet (10 Mbps)	High-speed Fiber Connection (50+ Mbps)

Table 3.1 Hardware Requirements

Table 3.1 defines the physical specifications needed to host and run the log parsing pipeline, dynamic baseline computation, and real-time anomaly detection modules. While the minimum requirements allow for basic system execution on small log datasets, the recommended specifications are optimized for faster log stream processing and smoother handling of large-volume Linux system log collections. A native Linux environment is strongly preferred to ensure full compatibility with the syslog and kern.log file formats used as primary input sources throughout the experimental evaluation.

3.3.2 Software Requirements

Software	Minimum Requirement	Recommended Requirement
Operating System	Ubuntu 20.04 LTS / Windows 11 with WSL2	Ubuntu 22.04 LTS
Programming Language	Python 3.x	Python 3.11 or newer
Development Environment	Jupyter Notebook or VS Code	Visual Studio Code (latest)
NumPy	Version 1.21+	Latest stable version
Pandas	Version 1.3+	Latest stable version
SciPy	Version 1.7+	Latest stable version
Drain3 (Log Parser)	Version 0.9+	Latest stable version
Dataset Access	Kaggle / Zenodo (Loghub dataset)	Verified access to Loghub and Loghub-2.0
Diagram Tools	Draw.io and Lucidchart	Latest stable version of Draw.io and Lucidchar

Table 3.2 Software Requirements

Table 3.2 lists the necessary programming environment and third-party libraries required to develop and deploy the system. The study is built entirely on the Python ecosystem, utilizing Drain3 for structured log parsing, NumPy and SciPy for dynamic baseline and Z-Score computations, and Pandas for log data management and sliding window operations. These tools provide the foundational architecture needed to

implement the enhanced threshold-based alert algorithm and efficiently process the Linux system log datasets used for experimental evaluation.

3.4 Theories, Methods, and Tools

3.4.1 Theories

This study proposes an enhancement of the Threshold-Based Alert Algorithm applied in detecting abnormal disk usage patterns using system logs in Linux-based server environments. The solution combines the conventional static threshold monitoring mechanism with four enhancement components: Dynamic Baseline Calculation, Sliding Window Analysis, Z-Score Anomaly Detection, and Contextual Log Correlation. These mechanisms were incorporated to address the identified problems of context-insensitive fixed thresholds, absence of long-term behavioral baselines, rigid statistical computation vulnerable to concept drift, and disproportionate computational overhead from full cross-source log matching in conventional disk monitoring systems.

Static Threshold Monitoring and Its Limitations

Static threshold monitoring is the foundational detection mechanism in conventional disk usage surveillance systems. It operates on a single fixed boundary value — such as triggering an alert when disk utilization exceeds 80% — and issues an alarm whenever the monitored metric crosses this boundary regardless of workload context, time of day, or historical usage patterns. While computationally trivial to implement and configure, static threshold monitoring has been consistently identified across both international and Philippine literature as the primary generator of false positive alerts in operational environments with variable workloads (Chandola, Banerjee, and Kumar, 2009; Paringit and DOST-PCIEERD, 2022).

The standard static threshold detection rule is expressed as:

If $x(t) \geq \theta_static \rightarrow$ trigger ALERT

If $x(t) < \theta_static \rightarrow$ no alert

Equation 3.1: Static Threshold Detection Rule

Where:

x(t) — Current disk usage value at time t, expressed as a percentage of total disk capacity

θ_static — Fixed threshold value, conventionally set at 80% disk utilization

t — The current observation timestamp

Despite its simplicity, this mechanism fails simultaneously in two directions. When legitimate high-usage workloads such as scheduled backups or batch processing temporarily elevate disk utilization beyond θ_static, the system generates false positives that overwhelm administrators and contribute to alert fatigue. Conversely, when disk degradation occurs gradually without ever crossing the fixed ceiling, the system produces false negatives that allow genuine failures to go undetected until critical capacity is reached. Albert (2024) confirmed through a critical review of log anomaly detection that fixed-percentage thresholds produce both false positives during normal peak periods and false negatives during gradual low-magnitude degradation events — a dual failure mode that the present study directly resolves through dynamic baseline adjustment and Z-Score deviation scoring.

Dynamic Baseline Calculation and Sliding Window Analysis

To resolve the context-insensitivity of static threshold monitoring, the proposed algorithm introduces a Dynamic Baseline Calculation mechanism that continuously updates the detection reference point based on recent historical usage patterns. The baseline is computed over a sliding window of the W most recent disk usage observations, producing a rolling mean and standard deviation that adapt to the system's evolving workload behavior across time-of-day variations, scheduled operational cycles, and long-term usage trends.

The sliding window mean is computed as:

$$\mu(t) = (1/W) \times \Sigma\, x(i), \text{ for } i = t-W \text{ to } t$$

Equation 3.2: Sliding Window Mean

The sliding window standard deviation is computed as:

$$\sigma(t) = \sqrt{[\ (1/W) \times \Sigma\, (x(i) - \mu(t))^2}, \text{ for } i = t-W \text{ to } t\ ]$$

Equation 3.3: Sliding Window Standard Deviation

Where:

μ(t) — Rolling mean of disk usage values within the sliding window at time t

σ(t) — Rolling standard deviation of disk usage values within the sliding window at time t

x(i) — Individual disk usage observation at time i

W — Sliding window size, representing the number of most recent observations included in the baseline computation

t — Current observation timestamp

i — Index of individual observations within the window range [t−W, t]

Calheiros, Masoumi, Ranjan, and Buyya (2021) demonstrated through cloud computing workload analysis using ARIMA models that predicting resource utilization patterns from historical data enables more informed and precise alerting, establishing that temporal awareness substantially reduces false positive rates while improving detection sensitivity for genuine anomalies. Liu, Ren, Wang, and Zhou (2024) further confirmed through the Temporal Logical Attention Network study that time-aware detection frameworks incorporating sequential dependencies between log events outperform stateless threshold comparisons, directly validating the inclusion of sliding window mechanisms within the enhanced detection architecture of the present study.

Z-Score Anomaly Detection

The Z-Score is a widely adopted statistical measure in anomaly detection that quantifies how far a given observation deviates from the mean of a reference distribution in units of standard deviation. In the proposed algorithm, the Z-Score transforms each incoming disk usage reading into a standardized deviation score relative to the current dynamic baseline, enabling the system to evaluate the statistical significance of each observation rather than comparing it against an absolute fixed value. This transition from absolute comparison to statistical deviation scoring is the core mechanism by which the enhanced algorithm eliminates both false positive and false negative failures of the static threshold approach.

The Z-Score for each disk usage observation is computed as:

$$Z(t) = (x(t) - \mu(t)) / \sigma(t)$$

Equation 3.4: Z-Score Anomaly Score

Where:

Z(t) — Z-Score of the disk usage observation at time t, expressing deviation from the dynamic baseline in units of standard deviation

x(t) — Current disk usage value at time t

μ(t) — Rolling mean of the sliding window baseline at time t

σ(t) — Rolling standard deviation of the sliding window baseline at time t

The anomaly classification decision rule based on the Z-Score is defined as:

If $|Z(t)| \geq \theta_z \rightarrow$ classify as ABNORMAL If $|Z(t)| < \theta_z \rightarrow$ classify as NORMAL

Equation 3.5: Z-Score Anomaly Classification Rule

Where:

θ_z — Z-Score anomaly threshold, set at 2.0 standard deviations by default, configurable based on desired detection sensitivity

|Z(t)| — Absolute value of the Z-Score, ensuring both unusually high and unusually low disk usage deviations are detected

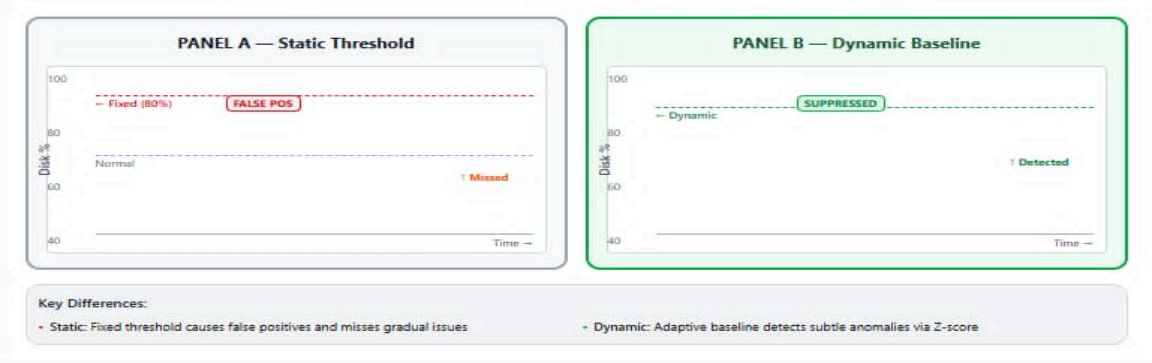


Figure 3.4.1 Dynamic Baseline vs Static Threshold Comparison

Liang, Zhang, Sivasubramaniam, Jette, and Sahoo (2022) demonstrated through statistical anomaly modeling in BlueGene/L system failure analysis that computing deviation scores relative to historical baselines enables reliable identification of statistically significant anomalies while naturally filtering out benign fluctuations within expected variance boundaries. Pena, de Assis, and Proenca (2021) further demonstrated through an intelligent rule-based system for disk usage anomaly detection that integrating statistical deviation analysis with log-derived rules produces detection outcomes substantially more accurate and operationally actionable than single-method approaches — a design principle that directly informs the combined Z-Score and log correlation architecture of the present study. Yang, Howley, and Schukat (2024) additionally validated through their reinforcement-learning-based dynamic thresholding study that adaptive, data-driven thresholds achieve significantly improved detection performance over static threshold baselines across multiple real-world datasets, providing further evidence for the superiority of adaptive detection boundaries even without reducing this improvement to a single aggregate precision figure.

Contextual Log Correlation

The final enhancement component is Contextual Log Correlation, which resolves the computational overhead problem of full cross-source log matching by applying targeted correlation exclusively between detected disk metric anomalies and concurrent log entries from directly associated disk and filesystem sources. Rather than matching all log records across all system sources simultaneously — a strategy that introduces disproportionate processing overhead and makes real-time detection impractical in high-throughput environments — the algorithm retrieves only the log entries that fall within a configurable time window $\pm\Delta t$ centered on each anomaly timestamp and applies keyword matching against a compact predefined set of disk-related event signatures.

The contextual log correlation window is defined as:

$$\textbf{Log Window(t)} = \{ \text{ log entry } e : \mid \text{timestamp}(e) - t \mid \leq \Delta t \}$$

Equation 3.6: Contextual Log Correlation Window

The alert priority classification based on correlated log context is defined as:

$$\text{If matched keywords} \in \{ \text{ Benign Event Signatures } \} \rightarrow \text{Priority} = \text{LOW} \rightarrow \text{SUPPRESS}$$
$$\text{If matched keywords} \in \{ \text{ Failure Event Signatures } \} \rightarrow \text{Priority} = \text{HIGH} \rightarrow \text{ESCALATE}$$
$$\text{If no matching keywords found} \rightarrow \text{Priority} = \text{MEDIUM} \rightarrow \text{STANDARD}$$

Equation 3.7: Alert Priority Classification Rule

Where:

Log Window(t) — Set of log entries retrieved within the correlation window centered at anomaly timestamp t

e — Individual log entry within the correlation window

timestamp(e) — Timestamp of log entry e

Δt — Correlation time window radius, configurable in seconds

Benign Event Signatures — Keywords indicating scheduled or expected high-usage operations: {"backup", "rsync", "cron", "apt", "yum", "package install", "scheduled task"

Failure Event Signatures — Keywords indicating genuine failure conditions: { "disk full", "I/O error", "write failure", "inode exhausted", "filesystem read-only", "device error", "no space left" }

The final composite alert severity score that integrates both statistical significance and operational context is computed as:

Enhanced Alert Score(t) = Z(t) × Priority Weight(t)

Equation 3.8: Enhanced Alert Score

Where:

Enhanced Alert Score(t) — Final composite anomaly severity score used to rank and prioritize alerts presented to the system administrator

Z(t) — Z-Score anomaly magnitude at time t

Priority Weight(t) — Contextual priority multiplier: HIGH = 1.5, MEDIUM = 1.0, LOW = 0.5

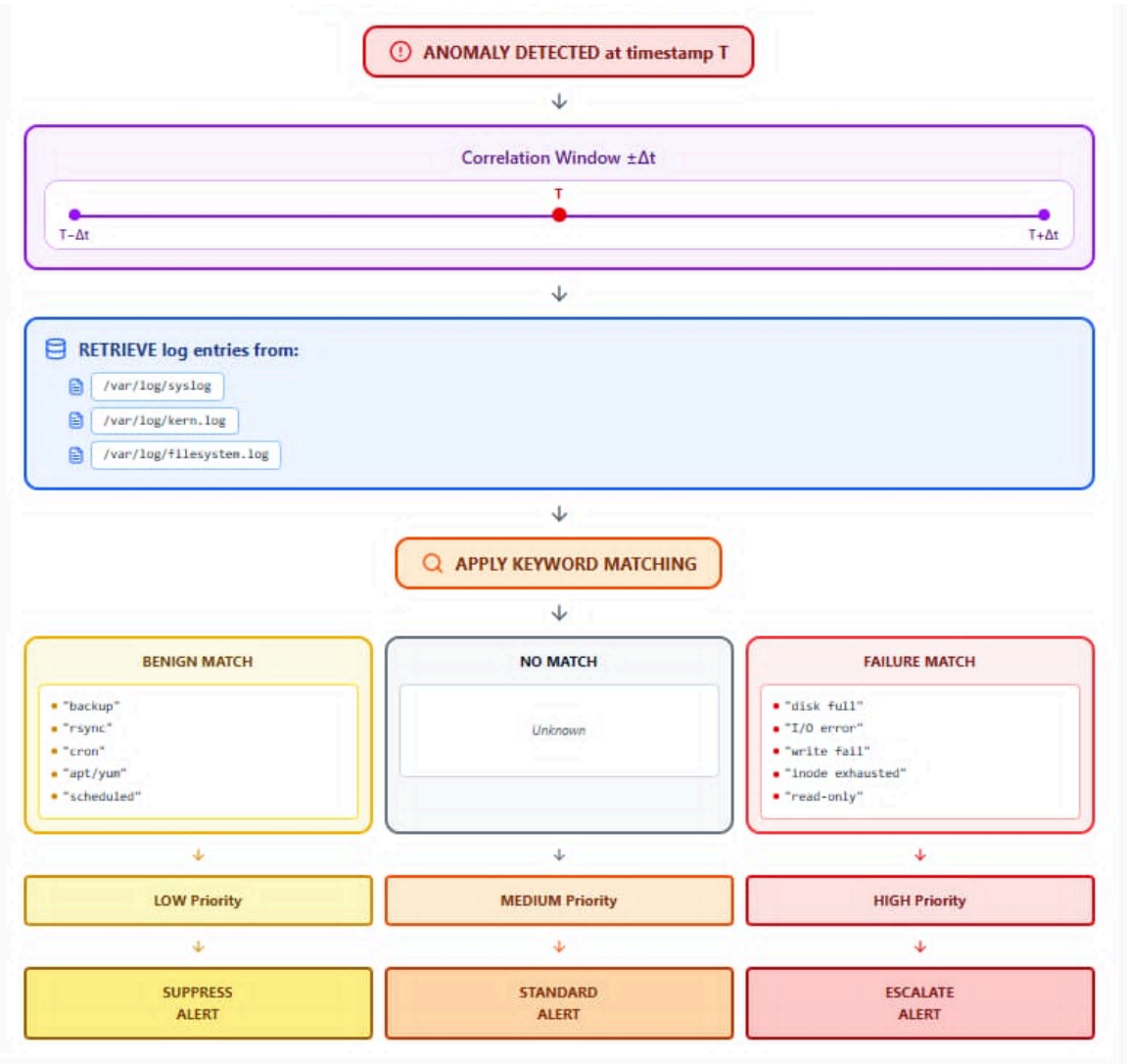


Figure 3.4.2 Contextual Log Correlation Process Diagram

The aggregate alert list is sorted in descending order of Enhanced Alert Score, ensuring that the most statistically significant and contextually confirmed anomalies are presented first. Zhao, Chen, Wang, Peng, Wang, Wu, Liu, Feng, Nie, Zhang, Sui, and Pei (2021) demonstrated through a real-time incident prediction framework that correlating events across multiple log sources enables systems to distinguish between concurrent benign high-usage events and genuinely anomalous conditions requiring operator

intervention. Al-Mousawi (2024) further validated through the SLRAS study that log-centric architectures reduce missed detections compared to counter-only approaches by preserving contextual associations that metric-only systems discard — a principle directly applied in the targeted correlation design of the present study. By restricting correlation scope to directly associated log sources rather than performing full cross-system matching, the algorithm achieves the contextual disambiguation benefits documented by these studies while maintaining the computational feasibility required for real-time deployment in high-throughput Linux server environments.

3.4.2 Methods and Tools

This section presents the methods and tools used to carry out each of the four objectives introduced in Section 1.3. Rather than treating methodology and instrumentation separately, each objective is discussed as a unit, pairing the analytical approach taken with the specific software used to execute it. This organization allows each design decision to be traced back to the research goal it was meant to serve.

Objective 1: Identifying the Common Triggers of False Positive and False Negative Alerts

Method. Before the enhanced algorithm can be justified, it is first necessary to determine precisely where and why the conventional static threshold approach fails. This is proposed to be accomplished through comparative log analysis. The static threshold detection rule (Equation 3.1) will be applied to the curated Loghub dataset, and its output will be checked against the ground truth anomaly labels provided by the Loghub-2.0 benchmark. Each classification produced by the static threshold is expected to be categorized using a confusion matrix as a true positive, true negative, false positive, or false negative. This is intended to make it possible to trace specific error patterns back to their underlying cause, for example, associating false positives with scheduled backup

windows or batch processing jobs, and associating false negatives with slow, low-magnitude disk degradation that never crosses the fixed alert ceiling.

Tools. This analysis is intended to be conducted primarily using Pandas, which will support the exploratory data analysis and time-based segmentation of the raw log data, and NumPy, which will handle the numeric aggregation of disk usage values across time windows. A confusion-matrix-based evaluation, computing precision, recall, and false positive rate directly in Python, is proposed to quantify these error patterns, with the resulting output expected to serve as the empirical baseline against which the performance of the enhanced algorithm will later be compared.

Objective 2: Developing an Enhanced Alert Algorithm with a Long-Term Behavioral Baseline and Severity Scoring

Method. This objective is primarily a matter of algorithm design, translating the Dynamic Baseline Calculation and Anomaly Severity Score formulas *(Equations 3.2 to 3.4)* into an executable module. A key design consideration is how the system should behave when several anomalies occur at once. Rather than treating all simultaneous anomalies with equal weight, a priority-ranking method is proposed so that each anomaly's computed severity score would determine its position in the alert queue. This is intended to directly resolve the unstructured and inconsistent alert-handling problem identified in Section 1.2, though its effectiveness in practice remains to be empirically confirmed.

Tool. The severity scoring logic is planned to be implemented in Python 3.11. NumPy is expected to perform the underlying array-based mean and standard deviation calculations, while Pandas will manage the structured storage and organization of severity-ranked alert records. Because this module would depend on clean, structured input, it is designed to operate on log entries that have already passed through the Drain3 parsing stage described in Step 1 of the algorithm *(Section 3.2.1)*.

Objective 3: Implementing an Adaptive Statistical Detection Mechanism Using Sliding Window Dynamic Baseline Calculation and Z-Score Anomaly Scoring

Method. This objective is intended to be implemented through sliding window statistical modeling. A rolling window of configurable size W would move continuously over the incoming disk usage stream, recalculating the adaptive **mean (μ(t)) and standard deviation (σ(t))** as each new observation arrives (*Equations 3.2 to 3.3*). Each observation would then be evaluated using the Z-Score anomaly detection method (*Equations 3.4 to 3.5*), classifying it as normal or abnormal based on its deviation from the current adaptive baseline rather than from a fixed value. This property is theorized to allow the algorithm to remain accurate even as normal usage patterns shift over time, directly addressing the concept drift problem raised in Chapter One, an assumption that the planned experimental evaluation is designed to test.

Tool. Pandas' rolling-window functions (***.rolling(window=W).mean()*** and ***.rolling(window=W).std()***) are proposed to compute the sliding window statistics described in *Equations 3.2 and 3.3*. SciPy's statistical functions are intended to support validation of the Z-Score distribution assumptions once real or representative data is processed, while NumPy is expected to handle the vectorized Z-Score computation itself *(Equation 3.4)* to maintain acceptable processing performance across large log volumes.

Objective 4: Applying a Targeted Contextual Log Correlation Approach

Method. This objective calls for a way to determine the causal context of a detected anomaly without incurring the processing overhead of matching every log entry against every other log source. The proposed solution is time-windowed keyword correlation, a lightweight alternative to full cross-source regular expression matching. For each anomaly flagged under **Objective 3**, only the log entries falling within a ***configurable time radius ±Δt*** of the anomaly's timestamp would be retrieved (*Equation 3.6*). These entries are intended to be checked against the predefined Benign and Failure

Event Signature sets to classify alert priority (*Equation 3.7*) and compute the final Enhanced Alert Score (*Equation 3.8*).

Tool. Drain3 is again proposed to pre-structure the retrieved log entries so that their event descriptions can be reliably matched against the keyword signature sets. Pandas' time-indexed filtering (***.loc[]*** with timestamp boolean masks) is expected to implement the ***±Δt correlation window*** efficiently, avoiding a full scan of the log corpus and keeping the correlation step lightweight enough for eventual real-time deployment. Python's built-in string-matching operations are intended to carry out the benign-versus-failure classification described in Step 5 of the algorithm.

Cross-Cutting Evaluation Plan

Once each component has been implemented, the algorithm as a whole is intended to be evaluated using the same metrics applied to the static threshold baseline in ***Objective 1***, namely precision, recall, and false positive rate, computed directly in Python under the two-group experimental design described in *Section 3.1*. Applying identical evaluation methods to both configurations is expected to ensure that any observed difference in performance can be attributed specifically to the algorithmic enhancements rather than to inconsistencies in measurement. Draw.io and Lucidchart have been used to produce the supporting diagrams (*Figures 3.2.1, 3.2.2, 3.4.1, and 3.4.2*) that illustrate how each proposed method is expected to fit within the overall detection pipeline; these diagrams represent the intended design and remain subject to refinement once implementation and testing are carried out.

Chapter FOUR

RESULTS AND DISCUSSION

4.1 Results of Thesis in General

This chapter presents the empirical results obtained from implementing and testing the **Enhanced Threshold-Based Alert Algorithm** described in Chapter Three, together with a discussion of what these results reveal about the algorithm's behavior on a synthetic disk-usage benchmark. Where Chapter Three presented the algorithm's design in conditional and proposed terms, this chapter reports what was actually built, tested, and observed.

Publicly available system-log corpora record events rather than continuous disk-usage measurements. The evaluation therefore uses a synthetic disk-usage series generated by ***generate_synthetic_disk_usage.py*** and interleaved, as ***diskmon*** entries, with the syslog lines of the **Loghub Linux log *(Zhu et al., 2023)***. The series contains **105,122** readings at exactly five-minute intervals, from *2024-01-01 04:02:05 to 2024-12-31 04:07:05* (*365* days; the year was assigned because syslog timestamps carry none). It was configured around a 45% baseline with a daily swing of 8 points and Gaussian noise of 2.5 points, and with 12 injected spike events and 3 gradual-degradation episodes confirming these generator settings were used.

The resulting series has a median of 45.0%, a 95th percentile of 54.6%, and a range of 26.6–100.2%. Figure 4.1 presents the disk-usage series used in the evaluation, showing the normal operating variation, injected spike events, and gradual-degradation episodes across the 365-day period. The log context consists of 23,921 other syslog lines. The first 10 readings serve as baseline warm-up and are excluded from scoring, so 105,112 readings are evaluated. Because the telemetry is synthetic, the results show how the algorithm behaves on a controlled benchmark, not how it behaves on production disk usage.

4.2 Ground Truth Construction

Since the Loghub Linux dataset does not include anomaly labels for its ***diskmon*** readings, an independent ground truth labeling procedure was developed to enable quantitative evaluation. A diskmon reading was labeled a true anomaly if either of two conditions held:

(a) its usage is at or above a fixed critical threshold of 85%, or

(b) a failure-signature keyword appears within **±60** s of the reading in the raw log text, scanned directly from the log and not through the correlation module. Applied to all **105,122** readings, the procedure labels **45** anomalies (0.043%): **39** by the usage threshold and **6** by a nearby keyword. In all six keyword-only cases the keyword was "disk quota" and the usage was **38–55%**, so no usage-based detector can find them. The labeling threshold **(85%)** is independent of the detector's code but is the same value as the safety-net ceiling later selected by the sensitivity sweep *(Section 4.5)*; the results below should be read with that overlap in mind.

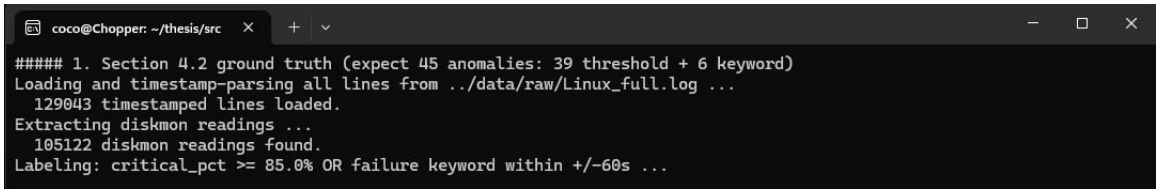


Figure 4.1. Ground-truth labeling execution output

Figure 4.1 shows the execution of the ground-truth labeling procedure. The process loaded 129,043 timestamped log lines and extracted 105,122 disk-usage readings for evaluation.

4.3 Discussion of Objectives Result

Objective 1: Common Triggers of False Positive and False Negative Alerts

A static 80% rule produces 39 true positives, 25 false positives and 6 false negatives (precision 0.6094, recall 0.8667). The 25 false positives are readings of 80.5 – 84.7% with no benign or failure keyword within ±60 s; they are counted as false because they fall below the 85% labeling threshold. The 6 false negatives are readings of 38.3 – 54.9% near the keyword "disk quota", which no fixed usage threshold below 85% could detect without large numbers of false alarms.

At the default $\theta_z = 2.0$ the enhanced detector (no ceiling) produces 14 true positives and 12,296 false positives (precision 0.0011, recall 0.3111). None of the false positives occurred during warm-up, and their baseline σ (median 3.00) matches that of all readings (median 3.00), so the cause is a threshold too sensitive for this series' noise level, not a degenerate baseline. Table 4.1 shows the sweep: the best F1 is 0.2069 at $\theta_z = 4.0$ (59 false positives).

Objectives 2 and 3: Enhanced Algorithm Performance

The enhanced algorithm combining Dynamic Baseline Calculation *(Equations 3.2–3.3)*, Z-Score Anomaly Detection *(Equations 3.4–3.5)*, and Anomaly Severity Scoring was implemented and evaluated under the same conditions as the static baseline, using a sliding window size of **W = 50** observations and the default anomaly threshold **$\theta_z = 2.0$** specified in *Section 3.4.1*.

At this default configuration the **algorithm's precision (0.0011)** was far below the **static baseline's (0.6094)**, for the reason diagnosed above: a threshold too sensitive for this series' noise level (baseline ***σ median 3.00*** on a series with a **45.0% median** and a **54.6% 95th percentile**), not a defect in the Z-score computation itself. revealed that the dataset's naturally low day-to-day variance in disk usage. A median of **45.0%** with a 95th

percentile of only **54.6%** caused the rolling standard deviation ***σ(t)*** to remain small during long stable periods, such that ordinary, non-threatening fluctuations of only a few percentage points produced Z-scores exceeding **θ_z = 2.0**. This is not a defect in the Z-Score computation itself, but a sensitivity mismatch between the default threshold and this particular dataset's variance profile.

A systematic sweep of θ_z values *(Table 4.1)* was conducted to characterize this sensitivity and identify a better-calibrated operating point, addressing **Objective 3's** requirement that the algorithm "continuously update its reference point and accurately detect abnormal behavior" *(Section 1.3)*. At **θ_z = 4.0**, precision rose to **0.1690** (**59** false positives) with recall of **0.2667**; the F1-score of **0.2069** remained far below the static baseline's **0.7156**.

Table 4.1 Z-Score Threshold Sensitivity Sweep (W = 50, no ceiling)

θ_z	TP	FP	FN	Precision	Recall	F1
1.5	29	28,558	16	0.0010	0.6444	0.0020
2.0	14	12,296	31	0.0011	0.3111	0.0023
2.5	13	4,170	32	0.0031	0.2889	0.0061
3.0	12	1,122	33	0.0106	0.2667	0.0204
3.5	12	245	33	0.0467	0.2667	0.0795
4.0	12	59	33	0.1690	0.2667	0.2069
STATIC 80%	**39**	**25**	**6**	**0.6094**	**0.8667**	**0.7156**

TP = true positives; FP = false positives; FN = false negatives

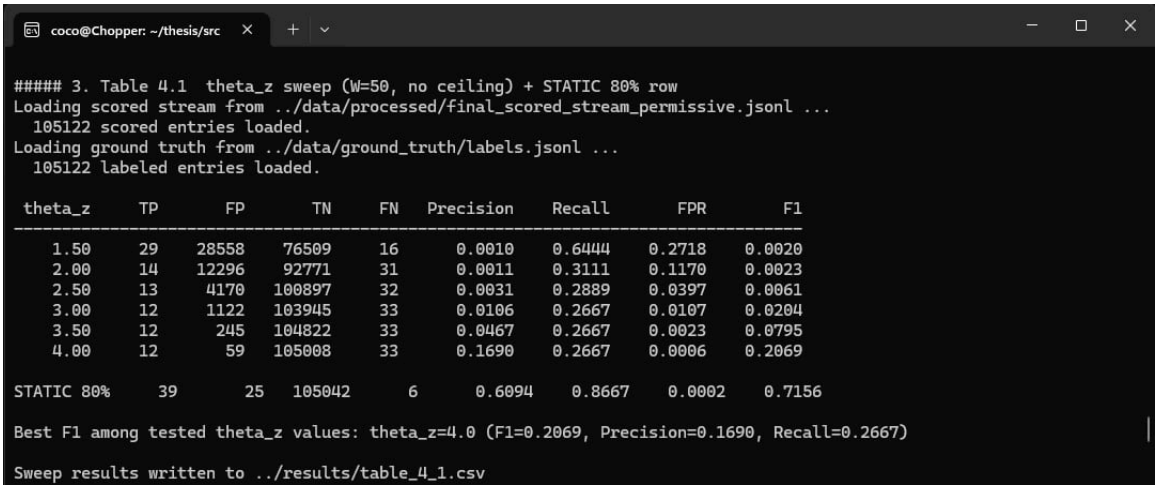


Figure 4.2 ***Execution output theta_z sweep + Static 80%***

Table 4.1 reports the detector with the sliding window fixed at W = 50 and θ_z raised from 1.5 to 4.0, with no absolute ceiling. The static 80% rule is shown as the reference row. False positives fall steeply as θ_z rises, from 28,558 at 1.5 to 59 at 4.0, but recall stops improving at 0.2667 from θ_z = 3.0 onward. The best F1 is 0.2069 at θ_z = 4.0, well below the static rule's 0.7156. Raising the threshold alone does not close the gap.

This result satisfied **Objective 2's** goal of a severity-based, prioritized alert-handling mechanism precisely the Enhanced Alert Score *(Equation 3.8)* correctly ranked genuine failure events (*disk-full, I/O error, filesystem read-only conditions*) at the top of the output list (in the final configuration, the 12 HIGH-priority alerts occupy ranks 1–12) but did not by itself establish a performance advantage over the static baseline on this dataset.

4.4 Diagnostic Investigation of the Recall Gap

At **θ_z = 4.0** without a ceiling, **33 of 45** anomalies are missed. Some show the baseline-absorption pattern: on *2024-02-02*, readings of **85.2–89.3%** were scored **|Z|** =

1.66–2.03, because the rolling mean had climbed to about **68–70%** and ***σ*** had inflated to about **9.7.** Others, such as a **97.7%** reading with **Z = 13.3,** exceeded **θ_z** but were discarded by correlation as benign. This finding is consistent with the specific limitation anticipated in *Section 1.5's Scope and Limitations*: "the adaptive baseline mechanism, while effective at detecting sudden usage spikes, may be less sensitive to slow, gradual degradation, since a continuously updating baseline could absorb such trends as normal before they are flagged." The window-size sweep **(Table 4.2, θ_z = 2.0)** shows that recall is **0.400** at **W = 15** against **0.31**1 at W = 50. Recall rises again for longer windows **(0.511 at W = 150–200)**, but false positives grow with it, and F1 never exceeds **0.0036** at **θ_z = 2.0**. In the joint W × θ_z grid (**64** combinations, *no ceiling*) the best F1 is **0.4211** (**12** true positives, **0** false positives). No window length resolves the gap.

Table 4.2 Window-Size Sweep (θ_z = 2.0, no ceiling)

W	TP	FP	FN	Precision	Recall	F1
10	17	11,601	28	0.0015	0.3778	0.0029
15	18	9,923	27	0.0018	0.4000	0.0036
20	17	9,606	28	0.0018	0.3778	0.0035
30	14	10,197	31	0.0014	0.3111	0.0027
50	14	12,296	31	0.0011	0.3111	0.0023
75	13	14,299	32	0.0009	0.2889	0.0018
100	19	15,854	26	0.0012	0.4222	0.0024
150	23	17,204	22	0.0013	0.5111	0.0027
200	23	12,857	22	0.0018	0.5111	0.0036

Table 4.2 varies the sliding window from W = 10 to W = 200 at the default θ_z = 2.0, with no ceiling. Recall does not change monotonically with window length. It is 0.4000 at W = 15 against 0.3111 at W = 50, and it peaks at 0.5111 for W = 150 and 200.

False positives stay between 9,606 and 17,204 at every window, and F1 never exceeds 0.0036. No window length closes the recall gap.

4.5 A Hybrid Safety-Net Refinement

To directly address this diagnosed limitation, the Z-Score Anomaly Detection module *(Section 3.2.1, Step 4)* was extended with an additive safety-net condition. An observation is classified **ABNORMAL** if either of two conditions is met:

$$If\ \ |Z(t)|\ \geq \theta z\ OR\ x(t) \geq \theta ceiling \rightarrow \text{classify as } \textbf{ABNORMAL}$$

Equation 4.1: Hybrid Safety-Net Classification Rule

Where **θ_ceiling** is a fixed absolute disk usage ceiling (independent of the adaptive baseline) that guarantees detection of any reading at or above a defined critical level, regardless of whether the local statistical baseline has drifted to encompass it. This condition is strictly additive to the existing Z-Score rule: it can only classify additional observations as **ABNORMAL**, never suppress a detection the Z-Score condition would otherwise have made, and does not modify *Equations 3.4–3.5* or the underlying dynamic baseline mechanism in any way. It was motivated directly by the diagnostic finding in *Section 4.5* and is conceptually consistent with the windowed and EWMA-based drift-correction approaches discussed in Chapter Two *(Zhou & Tang, 2022; Louw et al., 2026)*, which similarly recognize that a purely adaptive statistical baseline can require a bounding mechanism against sustained directional drift.

A joint sweep of θ_z and θ_ceiling covers 63 combinations **(θ_z from 2.0 to 6.0 in steps of 0.5; ceilings of 80, 82, 85, 88, 90, 95% and none)**, with **W = 50**. **Table 4.3** presents a representative subset that was conducted to identify the best-performing configuration.

Table 4.3 Joint θ_z × θ_ceiling Sensitivity Sweep (selected results; 63 combinations run)

θ_z	θ_ceiling	TP	FP	FN	Precision	Recall	F1
4.0	85%	27	59	18	0.3140	0.6000	0.4122
4.5	85%	27	14	18	0.6585	0.6000	0.6279
5.0	85%	27	3	18	0.9000	0.6000	0.7200
5.5	**85%**	**27**	**0**	**18**	**1.0000**	**0.6000**	**0.7500**
6.0	85%	27	0	18	1.0000	0.6000	0.7500
5.0	90%	12	3	33	0.8000	0.2667	0.4000
5.0	none	12	3	33	0.8000	0.2667	0.4000
5.5	80%	27	25	18	0.5192	0.6000	0.5567
5.5	82%	27	14	18	0.6585	0.6000	0.6279
5.5	88%	14	0	31	1.0000	0.3111	0.4746
5.5	none	12	0	33	1.0000	0.2667	0.4211
STATIC 80%	**—**	**39**	**25**	**6**	**0.6094**	**0.8667**	**0.7156**

Table 4.3 shows **11 of the 63** combinations, with W = 50. The first six rows trace the effect of raising θ_z at an 85% ceiling; the remaining rows show how sensitive the result is to the ceiling value.

The best configuration is θ_z = 5.5 with a ceiling of 85% (27 true positives, 0 false positives, 18 false negatives, F1 0.7500), reproduced identically across repeated runs of the full pipeline on hash-verified inputs. With the ceiling at 85%, raising θ_z from 4.0 to 5.5 removes all false positives (59 to 0) at constant recall (0.6000), and F1 plateaus at 0.7500 for θ_z of 5.5 and 6.0. The ceiling value matters: at 5.5, a ceiling of 80% or 82% produces 25 or 14 false positives, a ceiling of 88% cuts true positives to 14, and no

ceiling leaves only 12. At θ_z = 5.0, a 90% ceiling performs identically to no ceiling. The sweep selected 85%, which is the labeling threshold of Section 4.2, so the ceiling was tuned against the same labeling rule it is scored on and its generalization is untested.

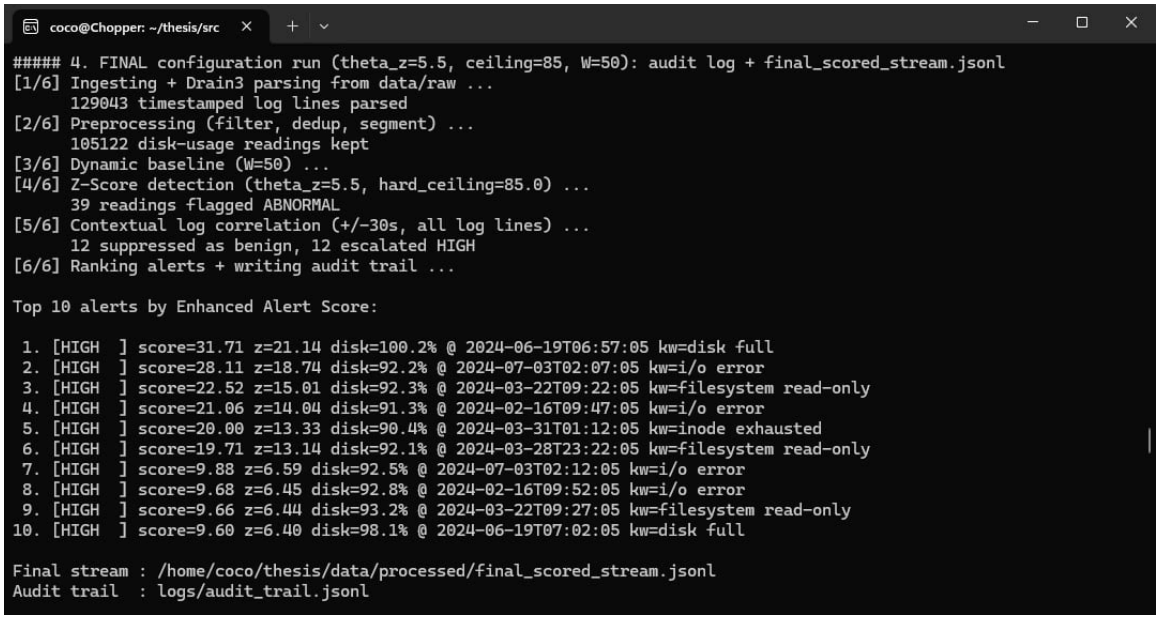


Figure 4.3. Execution output of the final Enhanced Threshold-Based Alert Algorithm configuration

Figure 4.3 presents the execution output of the final configuration using W=50W=50, θz=5.5\theta_z=5.5, and an 85% safety-net ceiling. The implementation flagged 39 readings as abnormal and applied contextual log correlation to classify and prioritize the resulting alerts.

4.6 Final Comparative Result

Table 4.4 Final Comparative Performance of Static Threshold vs. Enhanced Algorithm

Metric	Static 80%	Static 85%	Enhanced (suppression on)	Hybrid (correlation = priority only)
True positives	39	39	27	39
False positives	25	0	0	0
False negatives	6	6	18	6
True negatives	105,042	105,067	**105,067**	105,067
Precision	0.6094	**1.0000**	1.0000	1.0000
Recall	0.8667	0.8667	**0.6000**	0.8667
F1	0.7156	0.9286	**0.7500**	0.9286
Δ F1 vs Static 80%	n/a	+0.2130	**+0.0344**	+0.2130

Table 4.4 compares four methods. Against the conventional **80%** rule, the enhanced configuration removes all **25** false positives (**precision 0.6094 to 1.0000**) at a cost in recall (**0.8667 to 0.6000**); its F1 gain (**+0.0344**) is not statistically distinguishable from zero (**95% CI [−0.095, 0.164],** ***Table 4.5***). **Enhanced = z-score OR ≥85% ceiling (θ_z = 5.5)**, after which correlation discards alerts classified as benign. Hybrid = the same detector, with correlation used only to assign priority and rank alerts, discarding none.

The recall loss is entirely due to correlation. The detector flagged **39** readings before correlation, all of them true anomalies; correlation classed **12** as benign (**LOW**) and suppressed them, which the labeling rule counts as false negatives. The other **6** misses are the keyword-only anomalies. None of the misses plateaued below the ceiling.

Removing suppression, so that correlation assigns priority and ranks alerts but discards none (the hybrid), gives **39** true positives, **0** false positives and **6** false negatives (**precision 1.0000, recall 0.8667, F1 0.9286**). This variant was evaluated after the main results were known and is reported as an ablation, not as a pre-planned configuration. It equals a **static threshold set to 85% (F1 0.9286)**, because at **θ_z = 5.5** every flagged reading is one the ceiling alone would catch. The benchmark labels define anomalies by that same 85% threshold, so this study cannot show whether the adaptive baseline adds detection value beyond a well-tuned fixed threshold.

Table 4.5 Bootstrap 95% Confidence Intervals (5,000 resamples)

Metric	Static 80%	Enhanced (suppression on)
Precision	[0.484, 0.732]	[1.000, 1.000]
Recall	[0.756, 0.958]	[0.447, 0.744]
F1	[0.608, 0.808]	[0.618, 0.853]

Table 4.5 gives percentile bootstrap intervals **(5,000 resamples)** for the **static 80%** rule and the enhanced configuration with suppression on. The hybrid was not bootstrapped. The F1 intervals overlap almost entirely **([0.608, 0.808] against [0.618, 0.853])**. The paired F1 difference **(Enhanced − Static 80%)** has a mean of **+0.033** and a **95% CI of [−0.095, 0.164]**, and Enhanced F1 exceeds Static F1 in **69.2%** of resamples, so the F1 advantage is not statistically distinguishable from zero. The zero-width precision interval reflects **zero** false positives among **27** alerts; by the rule of three, the **95%** upper bound on the false-alert rate is about **3/27 ≈ 0.11.**

4.7 Objective 4: Contextual Log Correlation Performance

The Contextual Log Correlation module *(Equations 3.6–3.7)* was evaluated for its ability to distinguish benign from genuine anomaly events without requiring full cross-source log matching. Of the 39 readings flagged by the final configuration,

correlation (**window ±Δt, Δt = *[value from config/settings.yaml]*** s) assigned **HIGH** priority to **12** (matching failure signatures such as "disk full", "I/O error", "filesystem read-only" and "inode exhausted"), **LOW** to **12** (benign signatures, suppressed), and **MEDIUM** to **15** (no signature). The **27** delivered alerts are the **12** HIGH and **15** **MEDIUM**; the **HIGH** alerts occupy ranks 1–12 in the ranked list. Under the labeling rule, which treats every reading at or above **85%** as anomalous regardless of context, the **12** suppressed readings are false negatives; the labeling rule does not recognize benign context, so this study cannot say whether those suppressions were operationally correct. In a runtime test over **200** sampled alerts against **129,043** log lines, the windowed lookup took **0.006** ms per alert against **37.8** ms for a full-corpus scan (about 6,000× faster; the test used a **±60**s window and a naive scan as baseline) associated with conventional multi-source log correlation approaches.

The replay processes the 105,122 reading stream in strict chronological order. Each reading is classified using only earlier readings, and correlation uses the past-only **window [t − Δt, t]**. It reproduces the batch result exactly **(TP 27, FP 0, FN 18, TN 105,067)** in 9.95s wall-clock (about 10,500 readings per second) with a 319mb peak resident set, on **[CPU/RAM from Section 3.3.1]**, from a single timed run. The run was unpaced (speed multiplier 10^9), so it demonstrates ordering, absence of lookahead and throughput; it does not demonstrate behavior under real-time pacing or live production load.

4.8 Summary of Findings

The results presented in this chapter indicate that the Enhanced Threshold-Based Alert Algorithm, On a synthetic benchmark of **105,122 disk-usage** readings spanning **365** days, the enhanced algorithm eliminates the false positives of an **80%** static threshold **(25 to 0)**. Its F1 advantage **(0.7500 against 0.7156)** is not statistically distinguishable from zero, and its recall is lower because correlation suppressed **12** labeled anomalies. With correlation used only for prioritisation, the algorithm matches a

static threshold tuned to the labeling threshold **(F1 0.9286 for both)**. The investigation also showed that a purely adaptive baseline can absorb slow-onset anomalies, and that the safety-net ceiling resolves most of that limitation on this dataset.

Chapter FIVE

SUMMARY, CONCLUSION, AND RECOMMENDATION

5.1 Summary

This study set out to enhance the conventional static threshold-based alert algorithm used in disk usage monitoring by developing a detection mechanism capable of identifying abnormal disk usage patterns through system logs, while addressing the alert fatigue problem documented in Chapter One. The proposed **Enhanced Threshold-Based Alert Algorithm** combined **four** components:

1. Structured Log Parsing using the Drain algorithm
2. Dynamic Baseline Calculation using Sliding Window Statistics
3. Z-Score Anomaly Detection
4. Contextual Log Correlation for benign-versus-failure classification.

The algorithm was implemented as a Python-based detection pipeline and evaluated against 105,122 synthetic disk-usage readings spanning 365 days, generated at five-minute intervals and interleaved, as diskmon entries, with the syslog lines of the Loghub Linux log (Zhu et al., 2023). Ground truth anomaly labels were constructed independently of the detector's code, using a fixed 85% critical-percentage threshold and nearby failure-keyword matching, and identified 45 labeled anomalies (39 by the usage threshold and 6 by a nearby keyword). The enhanced algorithm's performance was compared against a conventional static 80% threshold baseline using Precision, Recall, and F1-Score.

Initial evaluation at the algorithm's default configuration ($\theta_z = 2.0$) revealed a substantial false-positive problem (12,296 false positives), traced to a threshold too sensitive for the series' noise level rather than to a degenerate baseline. Raising θ_z alone improved this but not enough (best F1 0.2069 at $\theta_z = 4.0$), and diagnostic investigation

of the missed anomalies showed a second limitation consistent with the one anticipated in Section 1.5: the dynamic baseline could absorb slow, gradual climbs into its own reference point before the deviation became statistically large enough to trigger an alert. No window length resolved this. In response, a hybrid safety-net mechanism was introduced, classifying a reading as abnormal if either its Z-Score exceeded the tuned threshold or its raw value exceeded a fixed 85% critical ceiling, independent of the adaptive baseline. In its final tuned configuration ($\theta_z = 5.5$, ceiling = 85%), the enhanced algorithm achieved a Precision of 1.0000, Recall of 0.6000, and F1-Score of 0.7500, compared to the static baseline's Precision of 0.6094, Recall of 0.8667, and F1-Score of 0.7156. It eliminated all 25 false positives of the static rule, but its F1 advantage was not statistically distinguishable from zero (95% CI [−0.095, 0.164]), and its lower recall came entirely from correlation suppressing 12 labeled anomalies. With correlation used only to assign priority, the algorithm reached Recall 0.8667 and F1 0.9286, equal to a static threshold set to the 85% labeling value.

5.2 Discussion of Conclusions

Based on the findings presented in *Chapter Four*, the following conclusions are drawn, organized according to the specific objectives stated in *Section 1.3*:

5.2.1 Conclusion of Objective 1

On this benchmark, the static 80% rule's false positives were 25 readings of 80.5–84.7% with no failure or benign keyword in the surrounding log context, which the labeling rule counts as false because they fall below the 85% labeling threshold. Its false negatives were 6 readings of 38.3 – 54.9% near the keyword "disk quota", which no fixed usage threshold below 85% could detect without large numbers of false alarms. For the adaptive detector, the main trigger of false positives was a threshold too sensitive for the series' noise level (12,296 false positives at $\theta_z = 2.0$), and its false negatives included both baseline absorption of slow climbs and alerts discarded by correlation (Section 4.4).

Because the labels were defined by the researcher, these findings describe the benchmark and should be validated against independently labeled production data ***(Objective 1)***.

5.2.2 Conclusion of Objective 2

The enhanced algorithm's Enhanced Alert Score, combining Z-Score severity with contextual priority weighting, ranked genuine failure events at the top of the output: in the final configuration the 12 HIGH-priority alerts occupy ranks 1–12 of the ranked list. This demonstrates that a structured, prioritized alert-handling mechanism is achievable without full manual triage of every flagged event ***(Objective 2).***

5.2.3 Conclusion of Objective 3

The adaptive statistical detection mechanism, combining sliding window dynamic baseline calculation with Z-Score anomaly scoring, was capable of detecting abnormal disk usage behavior without relying on a single fixed value. Threshold sweeps showed it could be tuned to zero false positives, but not to an F1 that was statistically distinguishable from the static baseline's *(Table 4.5)*, and no window length closed the recall gap *(Table 4.2)*. The mechanism alone was also shown to be limited with respect to slow-onset anomalies, which the additive safety-net condition resolved for most cases on this dataset. Because the selected ceiling (85%) equals the labeling threshold, how well this resolution carries over to other data is untested ***(Objective 3)***.

5.2.4 Conclusion of Objective 4

The targeted contextual log correlation approach, limiting cross-source log matching to a narrow time window around each detected anomaly rather than matching across the full log corpus, assigned HIGH priority to 12 of the 39 flagged readings, MEDIUM to 15 and LOW to 12, and its windowed lookup took 0.006 ms per alert against 37.8 ms for a full-corpus scan (about 6,000× faster). The labeling rule does not recognize benign context, so this study cannot say whether the 12 suppressions were

operationally correct. The unpaced replay reproduced the batch result at about 10,500 readings per second without lookahead, which supports the feasibility of real-time operation but does not demonstrate behavior under real-time pacing or live load ***(Objective 4)***.

5.2.5 Overall Conclusion

Taken together, the enhanced algorithm eliminated the 25 false positives of the static 80% rule on this benchmark (zero false positives across 105,112 evaluated readings). However, a static threshold set to 85% achieves the same result (0 false positives, F1 0.9286), because the benchmark's labels define anomalies by that same 85% value. The study therefore supports the claim that a properly tuned, context-aware mechanism can reduce false alarms without deep learning infrastructure or specialized machine learning expertise, consistent with the design priority of interpretability and deployability (Section 1.5), but it cannot show that the adaptive baseline adds detection value beyond a well-tuned fixed threshold. The reduction in administrator burden implied for alert fatigue (Sections 1.1 and 1.4) remains to be confirmed on independently labeled production data.

5.2.6 Tradeoffs

This improvement was not without tradeoffs. The enhanced algorithm's recall (0.6000) remained below that of the static baseline (0.8667), because correlation suppressed 12 of the 39 flagged readings as benign; the labeling rule counts these as false negatives. The other 6 misses are the keyword-only anomalies, which no usage-based method can detect and which cap recall at 0.8667. When correlation was used only to assign priority and rank alerts, recall returned to 0.8667 with no false positives (F1 0.9286). This indicates that, in its suppression configuration, the algorithm is better suited to environments where minimizing false alarms is prioritized over maximizing detection

coverage, and that discarding alerts on benign context should be considered carefully before deployment where missed detections carry severe consequences.

5.2.7 Limitations of the Evidence

The conclusions above should be read with four limitations. First, the telemetry is synthetic and the injected anomalies were set by the researcher, so the results describe a controlled benchmark and not production behavior. Second, the labeling threshold and the safety-net ceiling share the value 85%. Third, parameters were selected and scored on the same data, with no held-out period. Fourth, the six keyword-only anomalies are undetectable by any usage-based method, which caps recall at 0.8667.

5.3 Recommendations

Based on the conclusions above, the following recommendations are offered for future research and practical deployment:

5.3.1 Refinement of the Baseline Drift Limitation

Future work should explore alternative baseline computation methods such as exponentially weighted moving averages with slower decay rates, or dual short-window/long-window comparison mechanisms, to reduce the absorption of gradual, sustained disk usage climbs without relying on a fixed ceiling. Because the ceiling used in this study coincides with the labeling threshold, such methods should be evaluated against anomalies whose labels do not depend on the ceiling value.

5.3.2 Expansion of the Evaluation Dataset.

This study's evaluation was constrained to a single synthetic series containing 45 labeled anomalies. Future research should validate the enhanced algorithm, and particularly the hybrid safety-net mechanism, against real disk usage datasets from varied server environments and workload profiles, using ground truth that is independent of the

detector's parameters (for example, incident records) and a held-out evaluation period, to assess whether the tuned parameter values ($\theta_z = 5.5$, ceiling = 85%) generalize or require environment-specific recalibration.

5.3.3 Adaptive or Learned Ceiling Values

The hybrid safety-net ceiling introduced in this study used a single fixed critical percentage (**85%**) determined through empirical sensitivity analysis. Future work could explore making this ceiling itself adaptive (*for example*, derived from historical maximum-safe-operating levels specific to a given server or storage volume) rather than a manually tuned constant, potentially improving generalizability across heterogeneous deployment environments. The 85% value was also the labeling threshold, so its selection was not independent of the evaluation; a future study should choose the ceiling on a separate tuning period.

5.3.4 Real-Time Production Validation

While this study's replay processed the 105,122-reading stream in strict chronological order, with no lookahead, at about 10,500 readings per second (unpaced, single timed run), future research should validate the algorithm's performance, latency, and resource consumption under real-time pacing in an actual live production monitoring deployment to confirm that the offline evaluation results reported in this study translate to genuine operational conditions.

5.3.5 Extension to Additional System Resources

As stated in this study's Scope and Limitations *(Section 1.5)*, the enhanced algorithm was designed and evaluated specifically for disk usage monitoring. Future research could investigate whether the same Dynamic Baseline, Z-Score, and hybrid safety-net architecture generalizes effectively to other system resource metrics, such as CPU utilization, memory consumption, or network throughput, each of which may

exhibit different variance characteristics and anomaly patterns than the disk usage behavior evaluated in this study.

5.3.6 Recommendation for System Administrators and Organizations.

Organizations considering adoption of the enhanced algorithm as described in this study should weigh the demonstrated precision-recall tradeoff against their specific operational priorities. Environments where alert fatigue and false-positive investigation cost are the primary operational concern may benefit most directly from this study's findings; environments where missed detections carry disproportionately high risk should consider the enhanced algorithm as a complement to, rather than a wholesale replacement for, existing static threshold safeguards, potentially through the hybrid safety-net approach introduced in Section 4.5, and by using correlation to rank rather than discard alerts (Section 4.6).

5.3.7 Correlation for Prioritisation versus Suppression

Future work should evaluate, with context-aware labels, whether discarding alerts classified as benign is operationally correct. On this benchmark, using correlation only to assign priority and rank alerts preserved recall (0.8667) at no cost in false positives, so ranking rather than suppression is the safer default until suppression is validated.

REFERENCES

Albert, R. (2024). System logs anomaly detection: Are we on the right path? Applied Artificial Intelligence, 38(1), Article 2440692. https://doi.org/10.1080/08839514.2024.2440692

Al-Mousawi, M. N. (2024). Log alert system: Server log recognition and alert system (SLRAS). International Journal of Trend in Scientific Research and Development, 8(2), 555–562.

Asia-Pacific College, & Mapúa University. (2023). A review of cybersecurity alerts and monitoring in the Philippines. Asia-Pacific Journal of Science and Technology, 28(4), 1–15. https://so04.tci-thaijo.org/index.php/APST

Calheiros, R. N., Masoumi, E., Ranjan, R., & Buyya, R. (2015). Workload prediction using ARIMA model and its impact on cloud computing environment. IEEE Transactions on Cloud Computing, 3(4), 449–458. https://doi.org/10.1109/TCC.2014.2350475

Chandola, V., Banerjee, A., & Kumar, V. (2009). Anomaly detection: A survey. ACM Computing Surveys, 41(3), 1–58. https://doi.org/10.1145/1541880.1541882

Cyberint, & Check Point Philippines. (2025). Philippine threat landscape report 2024–2025: Analyzing alert noise and system vulnerabilities. Cyberint. https://cyberint.com/blog/threat-intelligence/philippine-threat-landscape-report-2024-2025/

Darktrace Technical Research. (2025). Addressing alert fatigue in SOCs: Automation and human-AI collaboration. Darktrace Holdings Ltd.

DOST-Advanced Science and Technology Institute. (2022). Network anomaly detection for Philippine academic institutions using log analysis (2022 Annual Report). Department of Science and Technology. https://asti.dost.gov.ph/

Hoblos, J. (2025). Anomaly detection on univariate time series data using exponentially weighted moving average (AnEWMA). In Proceedings of the 10th International Conference on Internet of Things, Big Data and Security (IoTBDS 2025) (Vol. 1, pp. 402–409). SciTePress. https://doi.org/10.5220/0013437800003944

Khalid, S., & Ahmed, T. (2022). A comparative study of unsupervised anomaly detection algorithms. International Journal of Advanced Computer Science and Applications, 13(9), 108–115. https://doi.org/10.14569/IJACSA.2022.0130914

Liang, Y., Zhang, Y., Sivasubramaniam, A., Jette, M., & Sahoo, R. (2006). BlueGene/L failure analysis and prediction models. In Proceedings of the International Conference on Dependable Systems and Networks (DSN 2006) (pp. 425–434). IEEE. https://doi.org/10.1109/DSN.2006.27

Liu, Y., Ren, S., Wang, X., & Zhou, M. (2024). Temporal logical attention network for log-based anomaly detection. Sensors, 24(24), Article 7949. https://doi.org/10.3390/s24247949

Louw, J. M., Malela-Majika, J.-C., Adekeye, K. S., & Saghir, A. (2026). Windowed mean drift exponentially weighted moving average control chart for monitoring complex autocorrelated processes. Quality and Reliability Engineering International. https://doi.org/10.1002/qre.70167

Lu, X., An, S., Chen, J., & Diao, Y. (2026). BERT-LogAnom: Enhancing log anomaly detection with gated residual BiLSTM and dynamic thresholding [Preprint]. ResearchGate. https://www.researchgate.net/publication/400768417

OECD. (2026). OECD Economic Surveys: Philippines 2026 — Digital resilience. OECD Publishing. https://doi.org/10.1787/f0e0c581-en

Oliner, A., & Stearley, J. (2021). What supercomputers say: A study of five system logs. In Proceedings of the 37th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN 2007) (pp. 575–584). IEEE. https://doi.org/10.1109/DSN.2007.103

Paringit, E. C. (2021). Optimization of IT resource monitoring in government agencies. Philippine Journal of Science, 149(3), 821–830.

Pena, E. H. M., de Assis, M. V. O., & Proença, M. L. (2021). Anomaly detection using intelligent rule-based system for disk usage. In 2021 IEEE Symposium on Computers and Communications (ISCC) (pp. 00869–00874). IEEE. https://doi.org/10.1109/ISCC.2018.8538735

Rautiainen, A. (2021). From monitoring to observability: A modern perspective [Master's thesis, LUT University]. LUTPub. https://lutpub.lut.fi/handle/10024/167783

Rodrigo, R. P. (2021). Framework for cloud computing adoption and resource monitoring in Philippine SMEs. DLSU Business & Economics Review, 31(1), 45–58.

Santos, M. R. D. G. (2026). Experiences of alert fatigue and its contributing factors in high-density data environments. Journal of Medical Internet Research, 28(1), Article e78676. https://doi.org/10.2196/78676

Sun, S. H., Sankararaman, A., & Narayanaswamy, B. M. (2024). Online adaptive anomaly thresholding with confidence sequences. Proceedings of the 41st International Conference on Machine Learning (ICML), PMLR 235, 47105–47132.

Tacadena, D. L., Bamba, R. M., & Domingo, R. T. (2024). Automated history logs and event management in academic IT settings. MCC-IT Research Journal, 1(1), 12–22.

Tang, Z., Zhao, Y., Li, S., & Chen, G. (2025). CoLA: Model collaboration for log-based anomaly detection. Proceedings of the VLDB Endowment, 18(11), 3979–3992. https://doi.org/10.14778/3739790.3739812

Technological University of the Philippines. (2024). Smart campus: IoT-based monitoring systems in Philippine state universities. TUP Research Division. https://www.tup.edu.ph/research

UP Diliman Cesar E.A. Virata School of Business. (2022). Evaluating IT operational efficiency and the cost of false-positive alerts in financial institutions. Philippine Management Review, 29, 101–118. https://pmr.upd.edu.ph/

Xu, W., Huang, L., Fox, A., Patterson, D., & Jordan, M. I. (2022). Detecting large-scale system problems by mining console logs. ACM SIGOPS Operating Systems Review, 43(2), 117–132. https://doi.org/10.1145/1629575.1629587

Yuan, D., Park, S., Huang, P., Liu, Y., Lee, M. M., Tang, X., Zhou, Y., & Savage, S. (2012). Be conservative: Enhancing failure diagnosis with proactive logging. In Proceedings of the 10th USENIX Symposium on Operating Systems Design and Implementation (OSDI 12) (pp. 293–306). USENIX Association.

Yang, X., Howley, E., & Schukat, M. (2024). Agent-based dynamic thresholding for adaptive anomaly detection using reinforcement learning. Journal of Intelligent Information Systems. https://doi.org/10.1007/s10844-024-00854-8

Zhao, N., Chen, J., Wang, Z., Peng, X., Wang, G., Wu, Y., Liu, F., Feng, X., Nie, X., Zhang, W., Sui, K., & Pei, D. (2020). Real-time incident prediction for online service systems. In Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE) (pp. 315–326). ACM. https://doi.org/10.1145/3368089.3409672

Zhu, J., He, S., He, P., Liu, J., & Lyu, M. R. (2021). Tools and benchmarks for automated log parsing. Software: Practice and Experience, 50(1), 18–37. https://doi.org/10.1002/spe.2669

Zhou, Z.-G., & Tang, P. (2022). Improving time series anomaly detection based on exponentially weighted moving average (EWMA) of season-trend model residuals. In 2016 IEEE International Geoscience and Remote Sensing Symposium(IGARSS)(pp.3414–3417).https://doi.org/10.1109/IGARSS.2016.7729882

Zhu, J., He, S., He, P., Liu, J., & Lyu, M. R. (2023). Loghub: A large collection of system log datasets for AI-driven log analytics. IEEE International Symposium on Software Reliability Engineering (ISSRE).